

Ch 4: Advanced Hashing Techniques

Data Structure and Algorithm Design

Dr. Udaya R. Dhungana
Assistant Professor
Pokhara University, Nepal
Former Guest Faculty, Darmstadt University of Applied Sciences, Germany

CamScanner



Overview

Ch 4: Advanced Hashing Techniques

4. Advanced Hashing Techniques (6 hrs)
   - 4.1. Hardware-Accelerated Hashing
   - 4.2. Perfect Hashing
   - 4.3. Cuckoo Hashing
   - 4.4. Hashing in Distributed Systems
   - 4.5. Case Studies in GPUs and FPGAs

CamScanner

Hashing

1. Review of Hashing System
2. Perfect Hashing
3. Cuckoo Hashing

CamScanner

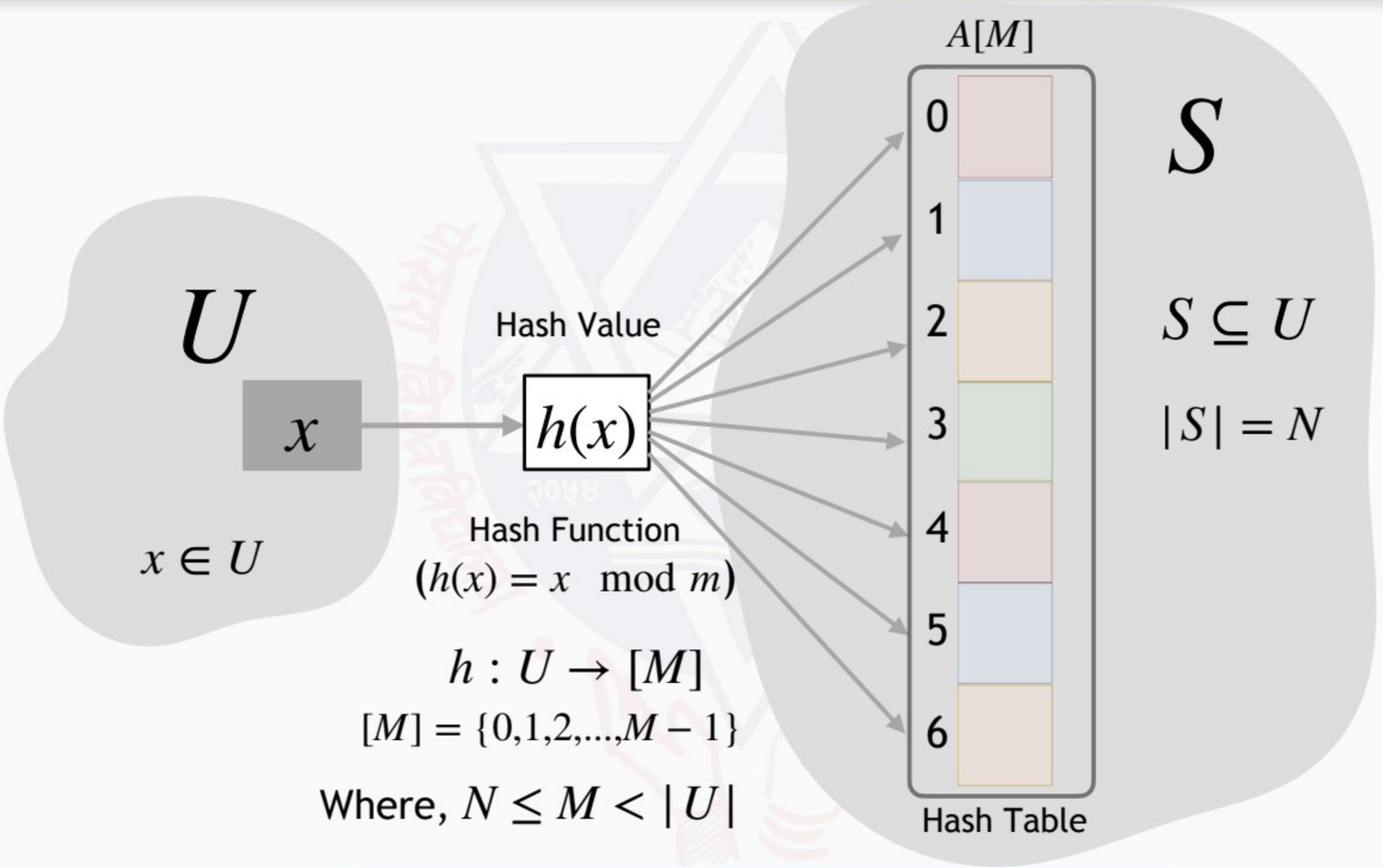
Hashing Basics

- Keys from universe U (think very large)
- Set $S \subseteq U$ of keys we actually care about (think relatively small) $|S| = N$
- Hash table A (array) of size M
- Hash function $h : U \rightarrow [M]$, where $[M] = \{0,1,2,3,...M-1\}$
- Idea: store x in $A[h(x)]$



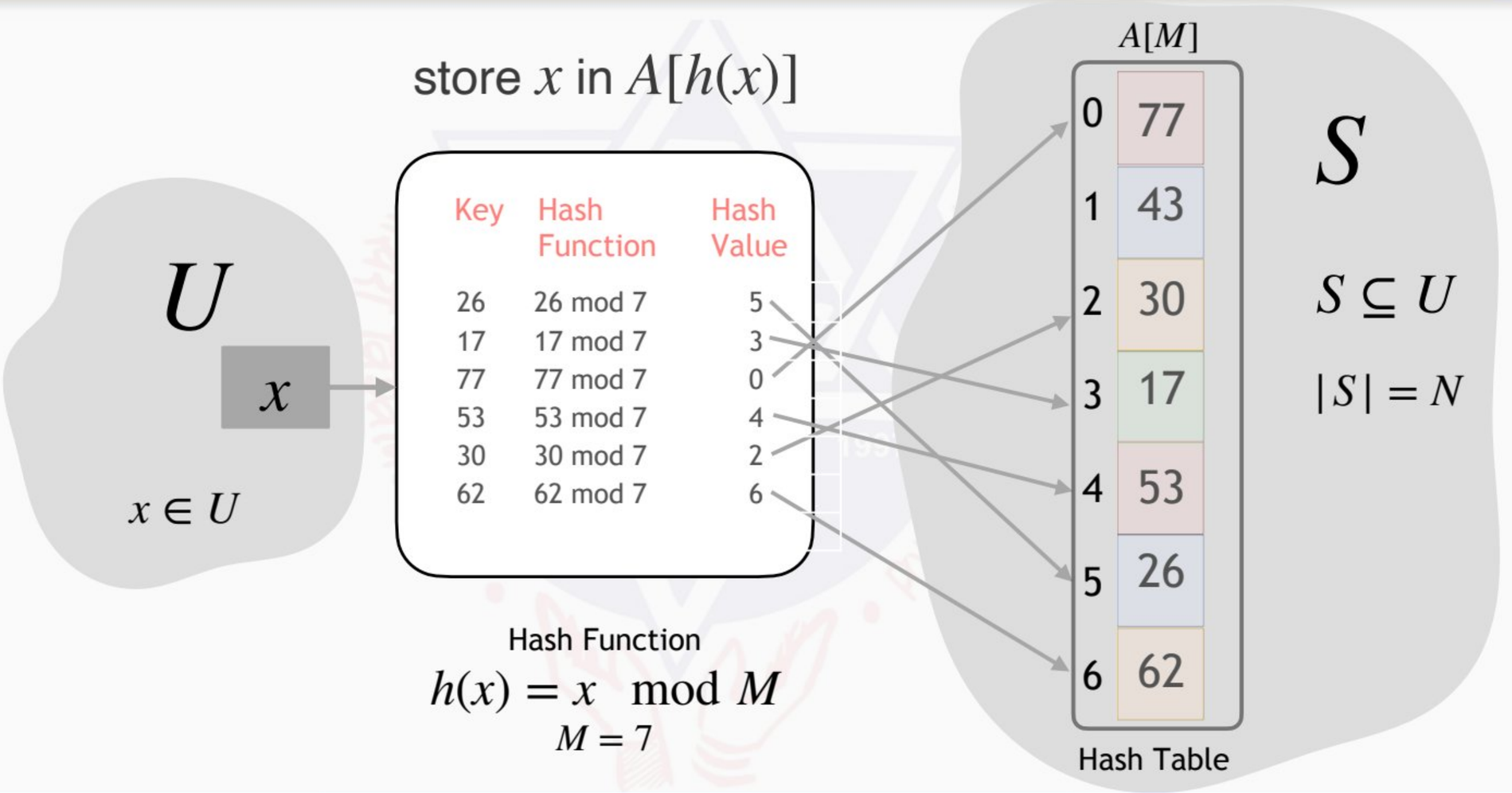
CamScanner

Hash System



CamScanner

Hash System


store x in $A[h(x)]$
U
x
$x \in U$
Key
Hash Function
Hash Value
26 26 mod 7 5
17 17 mod 7 3
77 77 mod 7 0
53 53 mod 7 4
30 30 mod 7 2
62 62 mod 7 6
Hash Function
$h(x) = x \mod M$
$M = 7$
$A[M]$
0 77
1 43
2 30
3 17
4 53
5 26
6 62
Hash Table
S
$S \subseteq U$
$|S| = N$

Hash Function

A simple hash function for integers:

```
//x is key of record and
//m is size of array
int h(int x, int m)
{
      return x % m;
}
```

A simple hash function for strings:

```
int h_ascii(string x, int m) {
   char ch[];
   ch = x.toCharArray();
   int xlength = x.length();

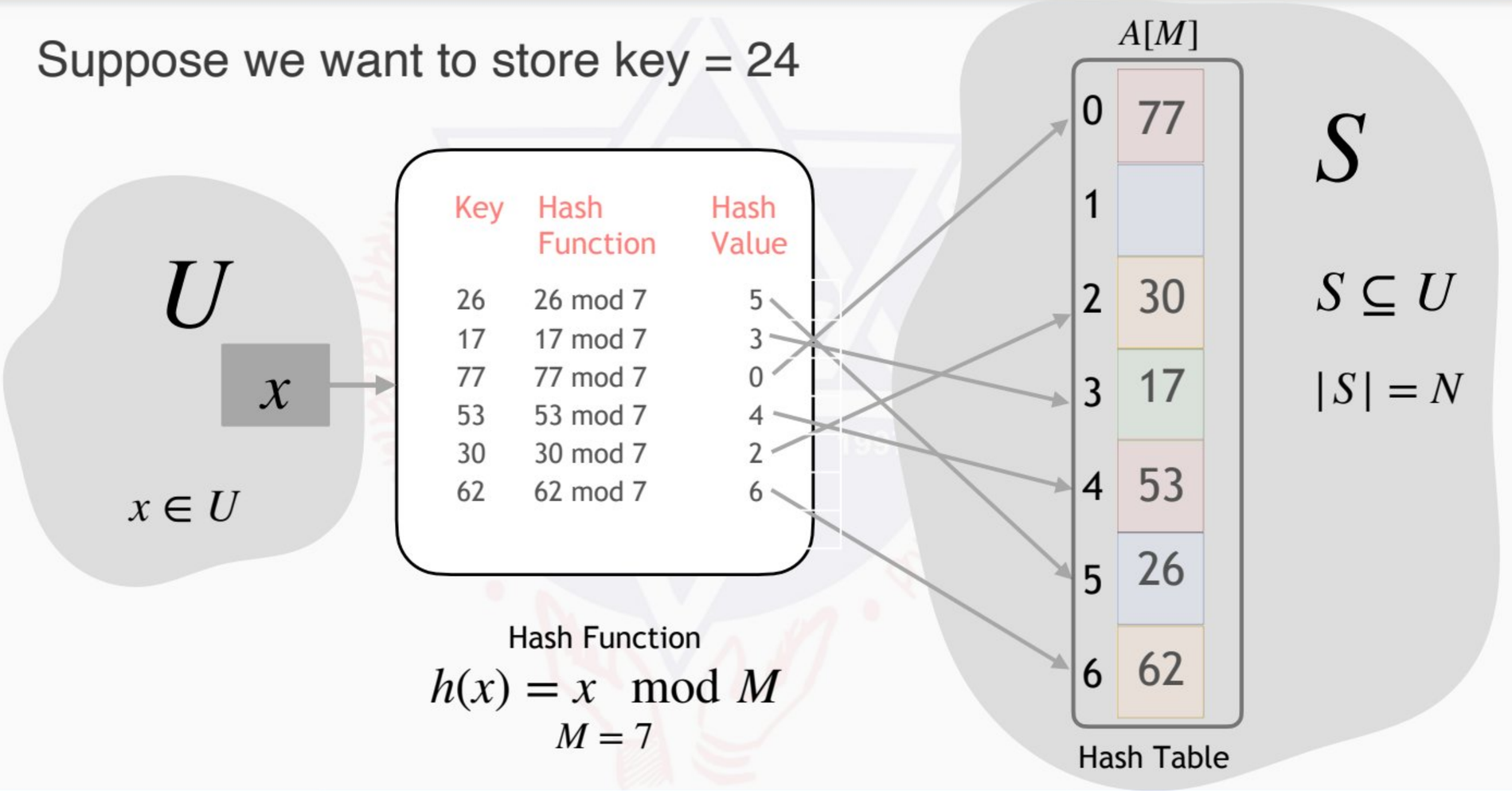
   int i, sum;
   for (sum=0, i=0; i < x.length(); i++)
      sum += ch[i];
   return sum % m;
}
```

- This function sums the ASCII values of the letters in a string and result is modulo-divided by size of the array, m.

CamScanner

Collisions in Hash System

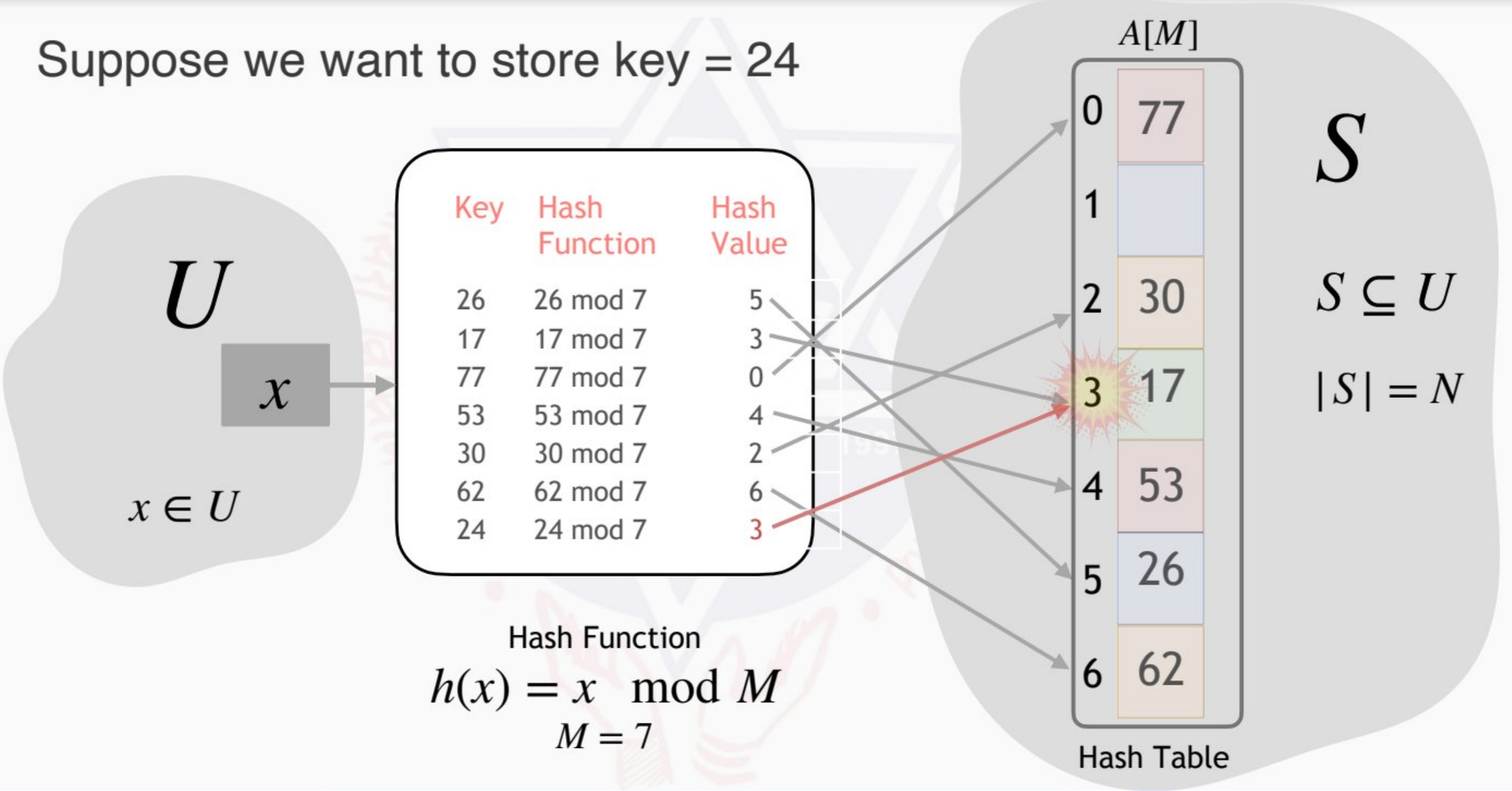
Suppose we want to store key = 24



CamScanner

Collisions in Hash System

Suppose we want to store key = 24



CamScanner

Collision Resolution Techniques

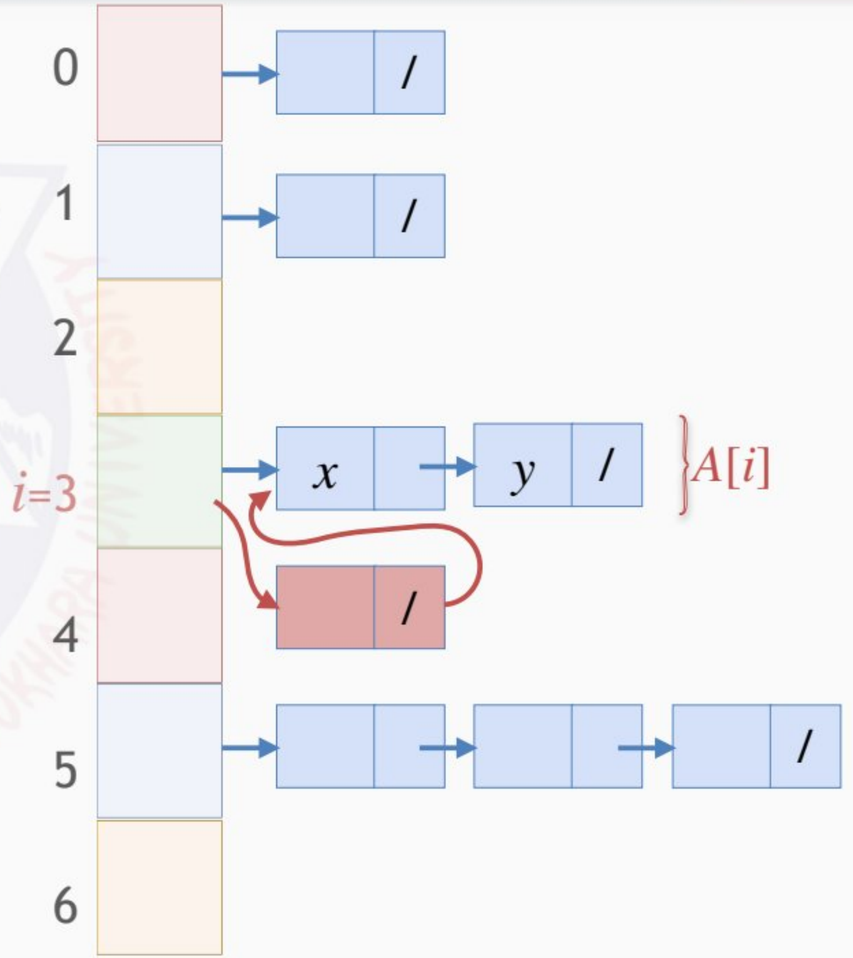
1. Open addressing (**Closed Hashing**)
    a) Linear probing
    b) Plus 3 probing
    c) Quadratic probing
    d) Double hashing
2. Closed Addressing (**Open Hashing**)
    a) Separate Chaining



CamScanner

Separate Chaining

- Collision resolution- Separate chaining
  - $A[i]$ is a linked list containing all x inserted where $h(x) = h(y) = i$.
- Operations:
  - $Search(x)$: Walk down the list at $A[h(x)]$ until we find x (or walk to the end of the list)
  - $Insert(x)$: Add x to the beginning of the list at $A[h(x)]$.
  - $Delete(x)$: Walk down the list at $A[h(x)]$ until we find x. Remove it from the list.



CamScanner

A Good Hash Function

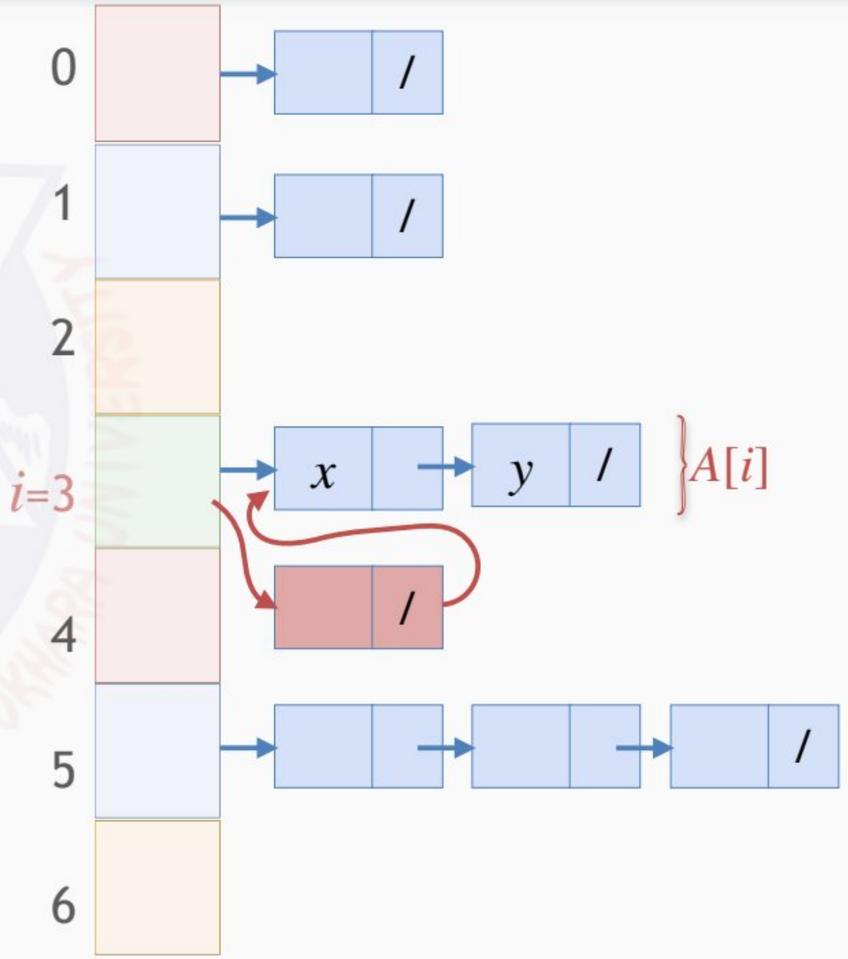
- What properties must a hash function have?



CamScanner

Hashing Basics

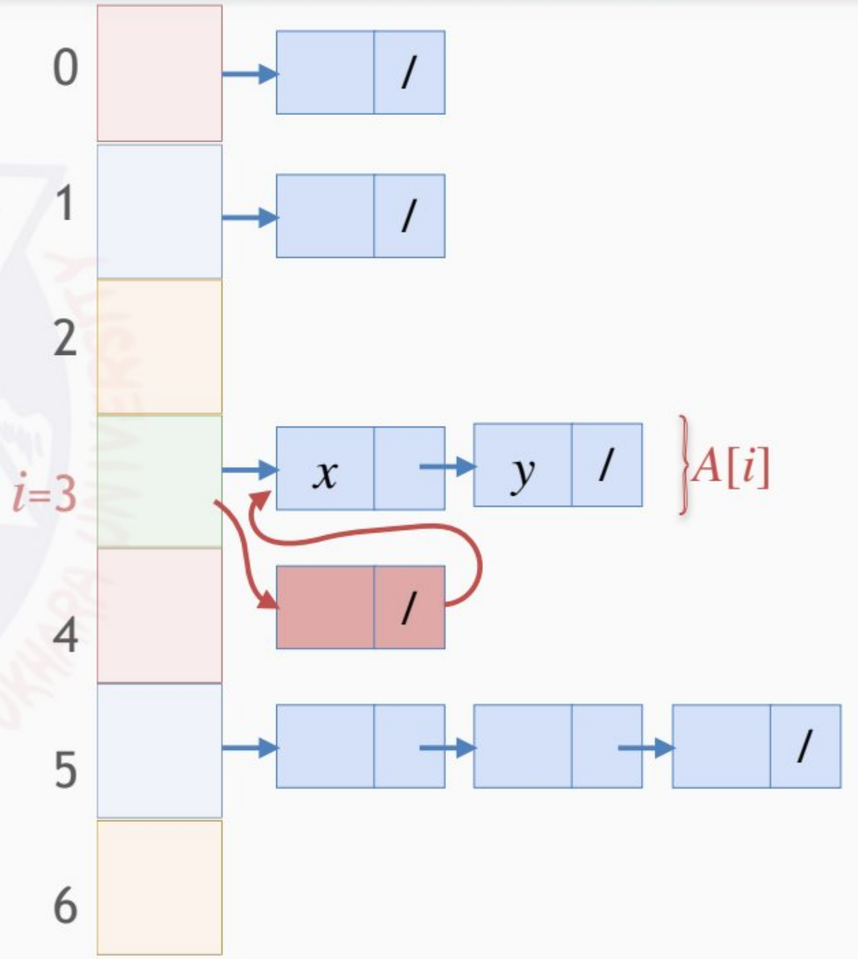
- ***Few Collisions***:
  - Time of search, delete for x is $O($length of list at $A[h(x)])$ and
  - insertion time for x = $O(?)$
- ***Small*** M (M is size of A)
- $h \rightarrow$ ***fast to compute***
  - $h : U \rightarrow [M]$



CamScanner

Hashing Basics

- Weakness of Hashing
  - For any choice of hash function
    - ∃ bad set of keys that all has to same slot
- Idea to deal with this
  - Chose hash function at random independent from keys.
  - A scheme that works is
    - ***Universal Hashing***



CamScanner

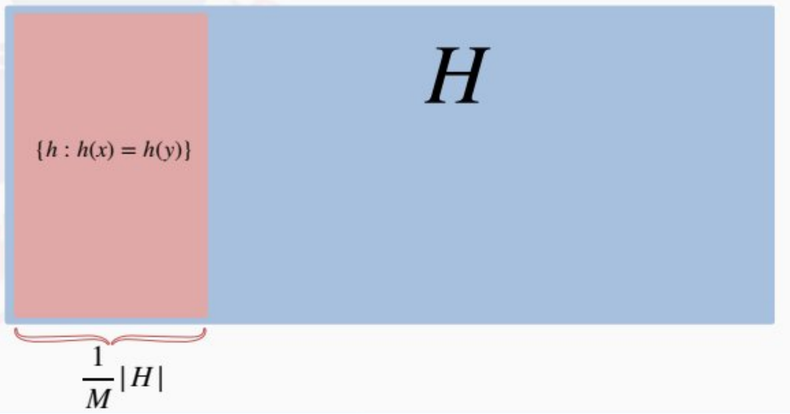
Universal Hashing

- Let U be a universe of keys and let H be a finite collection of hash functions mapping U to $\{0,1,2,3,...M-1\}$. The family of hash functions H is universal if for any two distinct keys x and $y \in U$,

$$|\{h \in H : h(x) = h(y)\}| = \frac{H}{M} \text{ where } x \neq y$$

- If h is chosen randomly from H, the probability of collision between x and y is at most $\frac{1}{M}$

i.e. $Pr_{h \in H}[h(x) = h(y)] \leq \frac{1}{M}$



Universal Hashing

- Choose h randomly from H. Suppose, hashing N keys into M slots in table T. Then for a given key x,

$$E[\text{number of collisions with } x] < \frac{N}{M}$$

Where $\frac{N}{M}$ is the load factor of the hash table T.



CamScanner

Constructing a Universal Hash Function

- **Let M be a prime:**
  - Choose M as a prime larger than the universe size $|U|$ can improve performance.
- **Key Decomposition:**
  - Decompose key k into $r+1$ digits. So
  - k is a vector and $k = < k_0, k_1, k_2 \ldots, k_r >$ where $0 \leq k_i \leq M-1$.
- **Hash Function Definition:**
  - Let a vector $a = < a_0, a_1, a_2 \ldots, a_r >$. Pick each a_i randomly from $\{0,1,2,3,...M-1\}$, ensuring randomness in the hashing process, Then the hash function $h_a(k)$ is
  - $h_a(k) = (\sum_{i=0}^{r} a_i \,.\, k_i) \; mod \; M$
- **Size of Hash Function Set:**
  - How big is set of hash function, $H = \{h_a\}$? (how many different hash function in this set??)
  - Since there are $r+1$ coefficients $a_0, a_1, a_2 \ldots, a_r$ in vector a, and each can take any value from $\{1,2,3,...,M\}$, the total number of hash functions in H is M^{r+1}.
  - $\therefore |H| = M^{r+1}$

CamScanner

Constructing a Universal Hash Function

- **Universality Condition:** ensures for any two different keys k and k', the probability of collision is bounded by:
  - $Pr[h_a(k) = h_a(k')] \leq \frac{1}{M}$
  - This condition is crucial to confirm whether the given function family is truly universal.
- **Prime M:**
  - The slide correctly assumes M is a prime number, which is useful for modular hashing.
  - However, in some cases, choosing M as a prime larger than the universe size can improve performance.

- **Why is a important?**
  - **Randomization**
    - Since a is chosen randomly, different instances of the hash function behave differently, preventing adversarial collisions.
    - Each different choice of a defines a unique hash function
    - The total number of possible functions is $|H| = M^{r+1}$
    - It ensures that the function family is large and provides ***good distribution properties***

CamScanner

Universal Hashing (in Summary)

- **Definition:**
  - Universal hashing is a technique used to select a hash function randomly from a family of hash functions to minimize the chances of collisions.
- **Randomized Hash function Selection:**
  - A hash function h is chosen randomly from a predefined universal family of hash function H.
- **Minimizing Collisions:**
  - A family of hash functions H is universal if for any two distinct keys x and y, the probability that they collide i.e. $h(x) = h(y)$ is at most $\frac{1}{M}$ where M is the number of possible hash values.
- **Mathematically**
  - $\forall x, y \in U, \quad x \neq y, \quad Pr_{h \in H}[h(x) = h(y)] \leq \frac{1}{M}$



CamScanner

Examples of Universal Hash Functions

- **Digit-wise Universal Hash Function (Polynomial Hashing) :**

$$h_a(k) = (\sum_{i=0}^{r} a_i \,.\, k_i) \; mod \; M$$

  - k is a vector and $k = < k_0, k_1, k_2 \ldots, k_r >$ where $0 \leq k_i \leq M - 1$.
  - A vector $a = < a_0, a_1, a_2 \ldots, a_r >$, each a_i randomly picked from $\{0,1,2,3,...M-1\}$
  - M is number of buckets

- **Linear Universal Hashing:**

$$h_{a,b}(k) = ((a \,.\, k + b) \; mod \; p) \; mod \; M$$

  - The key k is is treated as a single integer (not decomposed into digits).
  - Uses two random values: a is picked randomly from $\{1,2,3,..,p-1\}$ and b is picked randomly from $\{0,1,2,3,..,p-1\}$. By choosing a and b randomly, the hash function behaves unpredictably, making it hard for an enemy to force collisions.
  - The modulo operation with a prime number p ensures a good spread of hash values. The is p a prime number larger than the largest possible key.
  - M is number of buckets. The final modulo by M maps the result into the desired range.
  - widely used in hash tables and perfect hashing.

CamScanner

Implementation of Polynomial Universal Hashing

```cpp
#include <iostream>
#include <vector>
#include <cstdlib>  // For rand()
#include <ctime>    // For seeding rand()

using namespace std;

// Function to split a key into digits (assuming base M representation)
vector<int> splitKey(int key, int M) {
    vector<int> digits;
    while (key > 0) {
        digits.push_back(key % M);
        key /= M;
    }
    return digits;
}

// Universal hashing function
int universalHash(vector<int>& keyDigits, const vector<int>& a, int M) {
    int hashValue = 0;
    for (int i = 0; i < keyDigits.size(); i++) {
        hashValue = hashValue + (a[i] * keyDigits[i]);
    }
    return hashValue % M;
}
```


CamScanner

Implementation of Polynomial Universal Hashing

```
27  void insertIntoHashTable(vector<vector<int>>& hashTable, vector<int> keys, int M) {
28      // Seed the random number generator
29      srand(time(0));
30
31      for (int key : keys) {
32          vector<int> keyDigits = splitKey(key, M); // Convert key into digits
33          vector<int> a; // Random coefficients
34
35          // Generate random coefficients for hashing
36          for (int i = 0; i < keyDigits.size(); i++) {
37              a.push_back(1 + rand() % M); // Assigns random number from 1 to M
38              // a.push_back(rand() % M); // Assigns random number from 0 to M-1
39          }
40
41          // Compute the hash value
42          int hashValue = universalHash(keyDigits, a, M);
43
44          // Store in hash table
45          hashTable[hashValue].push_back(key);
46      }
47  }
```



CamScanner

Implementation of Polynomial Universal Hashing

```
49  // Function to search a key in the hash table
50  bool searchKey(vector<vector<int>>& hashTable, int key, int M) {
51      vector<int> keyDigits = splitKey(key, M);
52      for (int i = 0; i < hashTable[universalHash(keyDigits, {1, 2, 3}, M)].size(); i++) {
53          if (hashTable[universalHash(keyDigits, {1, 2, 3}, M)][i] == key) {
54              return true;
55          }
56      }
57      return false;
58  }
59
60  //print the elements of the hash table
61  void displayHashTable(vector<vector<int>>& hashTable) {
62      cout << "\nHash Table:\n";
63      for (int i=0; i<hashTable.size(); i++) {
64          cout << "Slot " << i << ": ";
65          for (int hashValue : hashTable[i]) {
66              cout << hashValue << " ";
67          }
68          cout << endl;
69      }
70  }
```


CamScanner

Implementation of Polynomial Universal Hashing

```
73  int main() {
74
75      int M = 97; // Prime number for modulo
76      /* Try different values of eg M = 97 and see how it affects the hash table
77          prime numbers are good choices for M
78          Example of prime numbers: 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59,
79          61, 67, 71, 73, 79, 83, 89, 97 */
80      vector<vector<int>> hashTable(M);
81
82      // Example keys
83      vector<int> keys = {123, 456, 789, 1023, 9876};
84
85      // Insert keys into the hash table
86      insertIntoHashTable(hashTable, keys, M);
87
88      // Display the hash table
89      displayHashTable(hashTable);
90
91      // Search for a key in the hash table
92      int key = 1024;
93      if (searchKey(hashTable, key, M)) {
94          cout << "\nKey " << key << " found in the hash table\n";
95      } else {
96          cout << "\nKey " << key << " not found in the hash table\n";
97      }
98
99      return 0;
100 }
```


CamScanner

Implementation of Linear Universal Hashing

Discuss the implementation of linear universal hashing using C++ in class



CamScanner

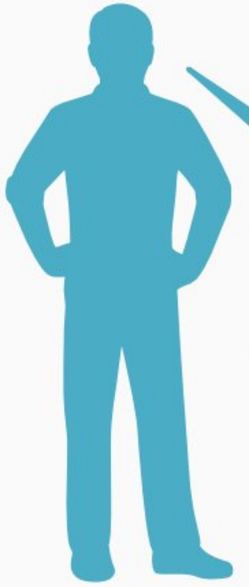
Analysis of Universal Hashing

- Universal hashing is a **probabilistic hashing technique** that ensures a **low probability of collisions** by randomly selecting hash functions from a **family of hash functions**.
- **Best Case**
  - ***No collisions***: If every key is hashed to a unique slot, operations take constant time.
  - Insertion/Search/Delete Cost: $O(1)$
- **Expected Case**
  - Few Collisions
  - Insertion/Search/Delete Cost: $O(1)$ on everage
- **Worst Case**
  - if all keys hash to the same bucket (though this is extremely rare in universal hashing).
  - Insertion/Search/Delete Cost: $O(n)$



CamScanner

Examples of Universal Hash Functions



Search and study some well-known families of universal hash functions.

CamScanner

2-Universal Hashing (Strongly Universal)

- A family of hash functions H is a ***2-universal*** if it satisfies two conditions:
  - **Universal Property:**
    - for all two distinct keys x and $y \in U, x \neq y$ and for randomly chosen function $h \in H$, the probability of collision is at most $\frac{1}{M}$.

$$Pr_{h \in H}[h(x) = h(y)] \leq \frac{1}{M}$$

  - **Uniform Distribution:**
    - for any fixed key x and any hash value v, the probability that a randomly chosen hash function $h \in H$ maps x to v is exactly $\frac{1}{M}$

$$Pr_{h \in H}[h(x) = v] = \frac{1}{M}$$

- This stronger condition ensures that collisions are minimized and hash values are uniformly distributed, making it useful in applications like cryptography and perfect hashing.

CamScanner

Universal vs 2-Universal Hashing

- Hash Tables
  - Universal hashing prevents worst-case scenarios where an attacker forces many collisions, ensuring good average-case performance
- Cryptography
  - Used in hash-based message authentication codes (HMAC) and signature schemes
- Data Structures
  - Applied in dynamic perfect hashing and cuckoo hashing
- Randomized Algorithms
  - Used in fingerprinting techniques and sketching algorithms.



CamScanner

Perfect Hashing

- Suppose I gave you a set of keys and I said just build a static table so that I can lookup whether the key is in the table with a good worst case time.
    - So I have a fixed set of keys- for example thousand of most common words in English (or you can imagine a dictionary) are given
- When I get a word I want to check whether the word is in the table.
- I would like to do that
    - Not with the expected performance (as before in universal hashing)
    - But with guaranteed worst case performance
- Is there a way of building it so that I can find this quickly with guaranteed worst case performance?
    - Yes, using Perfect Hashing!



CamScanner

Perfect Hashing

- This is used when the keys stored in hash table are expected to be static.
- This is how real dictionaries work.
  - Words are always fixed until the dictionary is updated
  - No insertions (or very few insertions during updating period)
  - No deletions
  - You most of time just read (lookup/ search) the words in a dictionary.
- It takes constant time $O(1)$ to search an item in the hash table (guaranteed worst case performance- order of one)



CamScanner

Perfect Hashing

- Given n keys.
  - Construct a static hash table (no insertion and deletions) of size $m = O(n)$ (I don't want a big table. I want it to be a table that is the size of my keys) such that search takes $O(1)$ time in the worst case.
  - Every single search I do should take order of one time.
- Idea:
  - Two-level scheme with universal hashing at both levels
  - When collision occurs, rather than using chaining, we are going to have another hash table there to do second hash such that there will be no collisions at level two.



CamScanner

Perfect Hashing

Perfect hashing is implemented using two level approach

- **Primary Level**
  - n keys are hashed into m slots in the hash table (**primary hash table**).
  - A hash function $h_1(x)$ from a universal family of hash functions is chosen to distribute keys into m slots
  - Collisions are handled at the **secondary level**.
- **Secondary Level**
  - Each slot in **primary hash table** gets its own hash table (**secondary hash table**)
  - For each slot i in **primary hash table**, a secondary hash table of size equal to the square of the number of collisions at the slot i is constructed to ensure no collision in the secondary level.
  - A new hash function $h_2(x)$ is chosen to store each colliding key in the secondary table.



CamScanner

Perfect Hashing

- **Primary Hash Function**

  Maps n keys are into m slots **primary hash table**.

  $h_1(key) = ((a_1 k + b_1) \% p) \% m$

  p is a prime number larger than any key

  m is number of slots in **primary hash table**.

- **Secondary Hash Function**

  Maps n keys are into m slots.

  $h_2(key) = ((a_2 k + b_2) \% p) \% m'$

  m' is the size of each **secondary hash table** for slot i in **primary hash table** where n' keys collide and $m' = (n')^2$.

CamScanner

Perfect Hashing

$$keys = \{10, 22, 37, 40, 52, 60, 70, 72, 75\};$$

Primary Level Hashing

$h_1 = ((a * key + b) \% p) \% M$

$a = 27\ b = 25$
$p = 101\ M = 9$

0	/
1	/
2	60
3	
4	22
5	
6	52
7	70
8	/

Secondary Level Hashing

$h_2 = ((a * key + b) \% p) \% M$

$a = 8\ b = 12\ p = 101\ M = 2^2 = 4$

0	1	2	3
10	/	75	/

0	1	2	3	4	5	6	7	8
/	40	37	/	/	/	72	/	/

$a = 94\ b = 59\ p = 101\ M = 3^3 = 9$

CamScanner

Implementation of Perfect Hashing (C++)

- Perfect Hashing Implementation
  - (with no collision handling in secondary hash table)
- Perfect Hashing Implementation
  - (with collision handling in secondary hash table)
- Discussion in Class



CamScanner

Not Use Perfect Hashing When …

- ***Small Datasets***
  - Overhead of computing perfect hash functions
- ***Dynamic Datasets***
  - If the key set changes frequently (insertions/deletions), rehashing is required, making it inefficient.
- ***Memory-constraint Systems***
  - May consume more memory due to two-level hashing.



CamScanner

Perfect Hashing

- ***Static Perfect Hashing***
- ***Dynamic Perfect Hashing***



CamScanner

Cuckoo Hashing

Cuckoo hashing mimics the way how cuckoo bird steals other bird's nest for laying



Cuckoo birds lay their eggs in other birds' nests

CamScanner

Cuckoo Hashing

Brood Parasitism

A cuckoo bird is a clever trickster in the bird world, known for its unusual breeding strategy where it lays its eggs in the nests of other birds, essentially "fooling" them into raising its chicks as their own, without building a nest or caring for its young itself

Female cuckoos carefully choose host birds and often lay eggs that closely resemble the color and pattern of the host's eggs, making it harder to spot the intruder.

Sometimes even removes one of the host's eggs to make room

Cuckoo



Cuckoo birds lay their eggs in other birds' nests

CamScanner

Cuckoo Hashing

- Cuckoo hashing is a **collision resolution technique** for hash tables that provides **constant-time lookups** in the worst case.
- Cuckoo hashing Idea
  - It has two or more arrays that store keys, where each array has its own hash function
  - It allows each key to reside in one slot of multiple hash tables
  - when a collision occurs, an item already in a slot is "kicked out" (like a cuckoo bird pushing another egg out of a nest), and the displaced item is relocated using its alternative hash function(s) in another hash table.



CamScanner

Cuckoo Hashing

- It was described by **Rasmus Pagh** and **Flemming Friche Rodler** in the year 2001. The paper was awarded the **European Symposium on Algorithms Test-of-Time award** in 2020.
- Cuckoo hashing is a type of closed hashing (Open Addressing).
- It uses two hash functions and two tables to avoid collisions.
- We pass our key to the first hash function to get a location in the first table. If that location is empty, we store the key and stop.
- If it isn't empty, we remove the old key stored there and store the current key in it.
- We then pass the old key to our second hash function to get a location in the second table. If we find that location empty, we store this old key there.
- If another key is already in that location in the second table, we remove it and store our old key from the first table.
- We then pass this removed key from table 2 to our first hash function to get a location in the first table.
- and so on… recursively …..



CamScanner

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$$h_1(key) = key \,\%\, 11$$



$$h_2(key) = (key/11) \,\%\, 11$$

CamScanner

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 20$

$h_1(20) = 20 \% 11 = 9$



$h_2(key) = (key/11) \% 11$

CamScanner

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 50$

$h_1(50) = 50 \% 11 = 6$



$h_2(key) = (key/11) \% 11$

CamScanner

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 53$

$h_1(53) = 53 \% 11 = 9$



$h_2(key) = (key/11) \% 11$

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 53$

$h_1(53) = 53 \% 11 = 9$



$h_2(key) = (key/11) \% 11$

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 53$

$h_1(53) = 53 \% 11 = 9$



$h_2(key) = (key/11) \% 11$

$key = 20$

$h_1(20) = (20/11) \% 11 = 1 \% 11 = 1$

CamScanner

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 53$

$h_1(53) = 53 \% 11 = 9$



$h_2(key) = (key/11) \% 11$

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 75$

$h_1(75) = 75 \% 11 = 9$



$h_2(key) = (key/11) \% 11$

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 75$

$h_1(75) = 75 \% 11 = 9$



$h_2(key) = (key/11) \% 11$

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \,\%\, 11$

$key = 75$

$h_1(75) = 75 \,\%\, 11 = 9$



$h_2(key) = (key/11) \,\%\, 11$

$key = 53$

$h_1(53) = (53/11) \,\%\, 11 = 4 \,\%\, 11 = 4$

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 75$

$h_1(75) = 75 \% 11 = 9$



$h_2(key) = (key/11) \% 11$

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 100$

$h_1(100) = 100 \% 11 = 1$



$h_2(key) = (key/11) \% 11$

CamScanner

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 67$

$h_1(67) = 67 \% 11 = 1$



$h_2(key) = (key/11) \% 11$

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 67$

$h_1(67) = 67 \% 11 = 1$



$h_2(key) = (key/11) \% 11$

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 67$

$h_1(67) = 67 \% 11 = 1$



$h_2(key) = (key/11) \% 11$

$key = 100$

$h_1(100) = (100/11) \% 11 = 9 \% 11 = 9$

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \,\%\, 11$

$key = 67$

$h_1(67) = 67 \,\%\, 11 = 1$



$h_2(key) = (key/11) \,\%\, 11$

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 105$

$h_1(105) = 105 \% 11 = 6$



$h_2(key) = (key/11) \% 11$

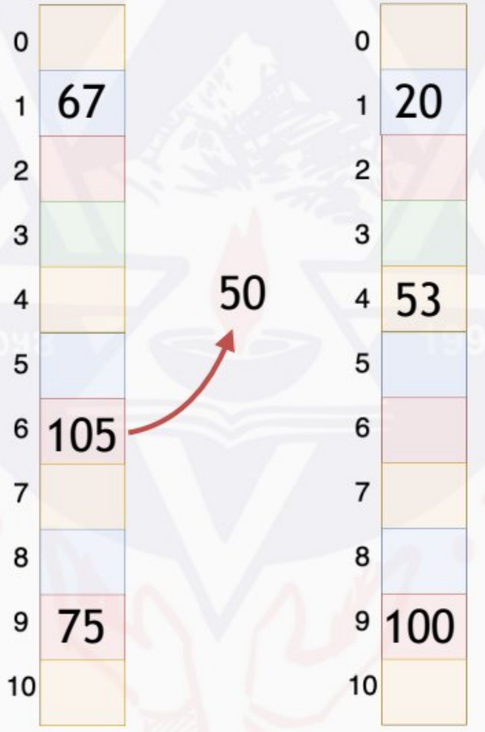
Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 105$

$h_1(105) = 105 \% 11 = 6$



$h_2(key) = (key/11) \% 11$

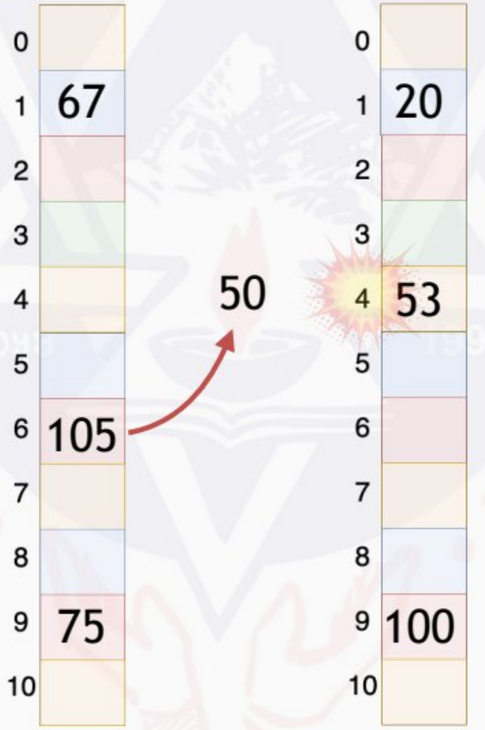
Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 105$

$h_1(105) = 105 \% 11 = 6$



$h_2(key) = (key/11) \% 11$

$key = 50$

$h_1(50) = (50/11) \% 11 = 4 \% 11 = 4$

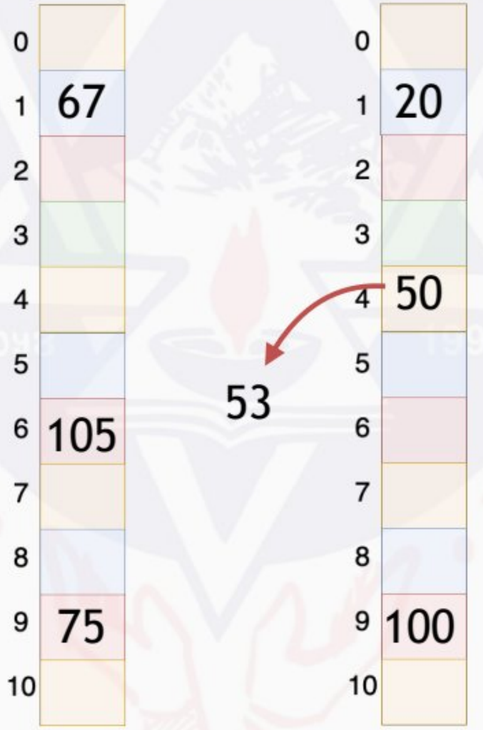
Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 105$

$h_1(105) = 105 \% 11 = 6$



$h_2(key) = (key/11) \% 11$

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 105$

$h_1(105) = 105 \% 11 = 6$

$key = 53$

$h_1(53) = 53 \% 11 = 9$



$h_2(key) = (key/11) \% 11$

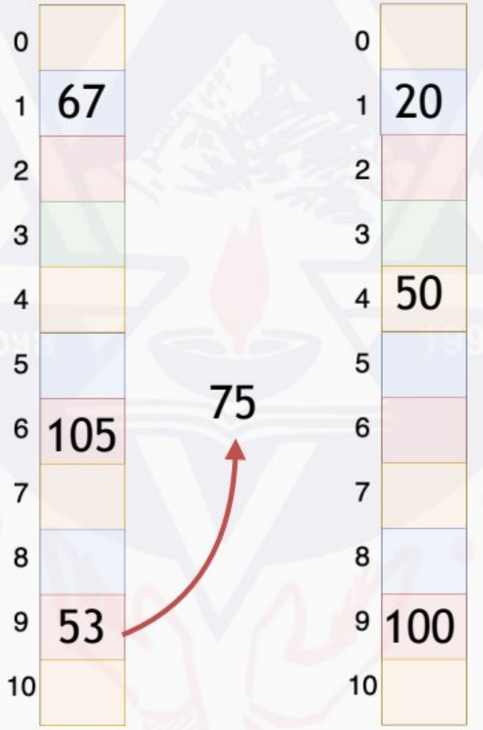
Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 105$

$h_1(105) = 105 \% 11 = 6$



$h_2(key) = (key/11) \% 11$

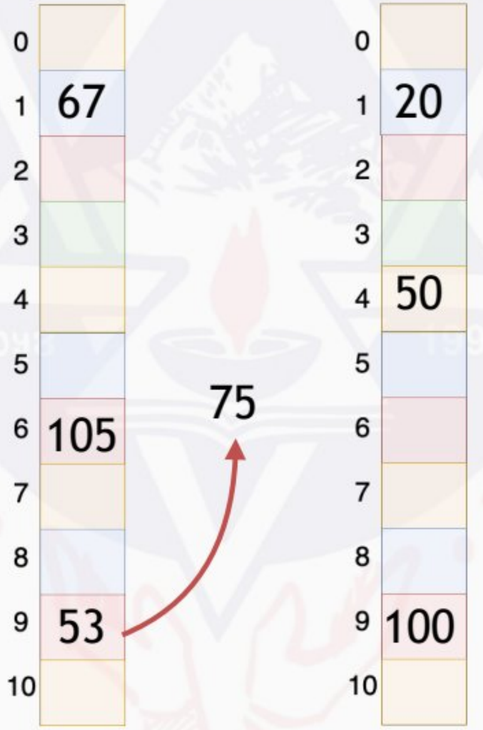
Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 105$

$h_1(105) = 105 \% 11 = 6$



$h_2(key) = (key/11) \% 11$

$key = 75$

$h_1(75) = (75/11) \% 11 = 6 \% 11 = 6$

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

$h_1(key) = key \% 11$

$key = 105$

$h_1(105) = 105 \% 11 = 6$



$h_2(key) = (key/11) \% 11$

Cuckoo Hashing

Key = {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}

and so on …

Complete by yourself …

Analysis of Cuckoo Hashing

- Insertion
  - Best case: $O(1)$ if no displacement
  - Worst case: $O(n)$
    - If too many collisions occur (cycles in displacement), we need to **rehash** the table with time $O(n)$
- Search
  - Worst/Best case: $O(1)$
    - Since each key is stored in at most **two locations**, we only need to check **two positions**, making lookup $O(1)$ **in the worst case**.
- Deletion
  - Worst/Best case: $O(1)$
    - Deletion is simple: remove the key from one of its two possible locations in $O(1)$ time



CamScanner

Cuckoo Hashing

- Advantages
  - Constant-time worst-case lookup O (1)
  - Less wasted space compared to chaining
  - Parallel lookups (since multiple locations exist)
  - Avoids long linked lists (unlike chaining)
  - High cache efficiency (due to direct access to memory)
- Disadvantages
  - Potential for infinite loops (resolved using rehashing)
  - More complex insertions than simple chaining
  - Needs extra space for multiple hash tables
- Applications
  - High-speed network routers
  - Database indexing
  - Cache implementations
  - Memory-efficient data structures

CamScanner

Hashing in Distributed Systems

- Web Caching
- Consistent Hashing
- Distributed Hash Systems
- Distributed Hash Table
- Case Studies

CamScanner

Hashing in Distributed System

- Hashing has a lots of applications in solving various real world problems
- Whether it is ***Peer-to-Peer System*** or ***Distributed System***
  - Need to distribute the loads among various nodes (machines)
- Load distribution can be done by
  - Hashing technique- ***consistent hashing***
- Consistent Hashing
  - original motivation: ***Web caching***
  - By Karger, Lehman et. al. in 1997 (see their paper here: https://www.cs.princeton.edu/courses/archive/fall09/cos518/papers/chash.pdf)



CamScanner

Web Caching

- Imagine a browser requesting a URL, like ***amazon.com***.
- One could request the page from the appropriate Web server
  - ***Problem***: if the page is requested over and over again, it's wasteful to repeatedly download it from the server.
  - ***Solution***: use a Web cache
    - which stores a local copy of recently visited pages
    - When a URL is requested, local cache is checked first.
      - If the page is in local cache, send it directly to the browser- *no need to contact the original server*
      - If it is not in the local cache, it is downloaded from the suitable server and the result is sent to both browser and local cache for future use
- Caching is good
  - much faster response time
  - Fewer request to far away servers- less network traffic, less congestion in the queues at network switches



CamScanner

Where to place Web Cache?

- ***Individual web cache***
  - At each end user machine
- ***Shared web cache***
  - Implement a web cache that is shared by many users



CamScanner

Shared Web Cache

- ***Problem***:
  - aggregated cache might be very large
  - So it might not fit on a single machine
- ***Solution***:
  - **IDEA**: *cache over multiple machines*
  - implement a shared cache at a large scale by spreading the cache over multiple machines



CamScanner

Shared Web Cache

- Suppose now
  - a browser requests a web page- ***amazon.com***
  - the browser wants to find the web page at local web cache
- Also, suppose
  - the shared cache is spread over 100 machines
  - ***Problem***: Where should you look for a cached copy of the Web page?



CamScanner

Shared Web Cache

- A bad solution:
  - You could poll all 100 caches for a copy
  - But it is infeasible:
    - Large number of users and caches
- Good Solution
  - **Hashing**
    - *mapping from URLs to caches*

*(Instruction to students- perform a detail study on a **MD5** , a well known hash function)*



CamScanner

Mapping URLs to Caches using Simple Hashing

- Suppose
  - there are n caches named $\{0,1,2,3...,n-1\}$,
  - x is the URL of a web page
- Then store the web page x at the cache server named

  $$h(x) \bmod n$$

- $h(x)$ is probably something like a 32-bit value, representing an integer (2^{32}) that is very bigger than n — so we apply the "***mod n***" operation to recover the name of one of the caches.



CamScanner

Mapping URLs to Caches using Simple Hashing

- Store the web page x at the cache server named

$$h(x) \bmod n$$

Problems ?



CamScanner

Mapping URLs to Caches using Simple Hashing

- Store the web page x at the cache server named

  $$h(x) \ mod \ n$$

- Problem:
  - n caches are not static
  - A new Web cache server can be added i.e. $n + 1$
  - Or an existing can fail or loose connection i.e. $n - 1$
  - Web cache servers are changing over time



CamScanner

Mapping URLs to Caches using Simple Hashing

- What will happen if n changes in $h(x) \bmod n$?
  - For an object x.
    - $h(x) \bmod n \neq h(x) \bmod n + 1$
  - So changing n forces almost all objects to relocate (rehashing).
    - rehashing cost = ? (For very large data)
    - This is a disaster for applications where n constantly changes
  - Solution
    - **Consistent Hashing**
    - Goal: almost all objects stay assigned to the same cache even as the number n of caches changes



CamScanner

Consistent Hashing

- Key Idea:
  - Apply Hashing for both
    - names of objects (URLs) x (as before) - $h(x)$
    - names of cache servers s (in addition) - $h(s)$
  - Both need to be hashed to the same range.



CamScanner

Consistent Hashing

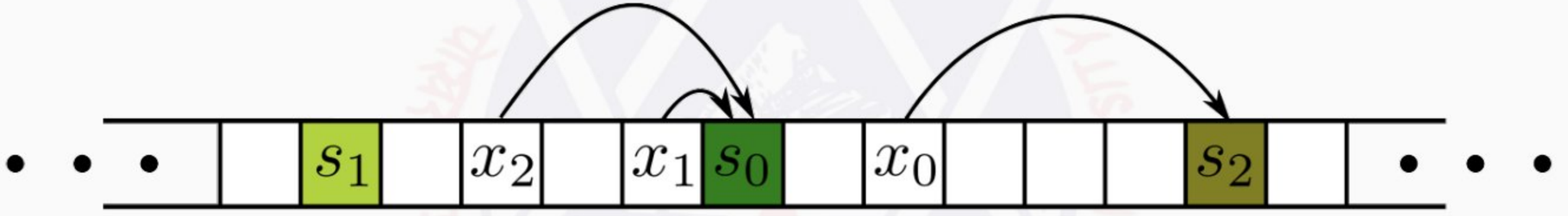
- Which objects x are assigned to which caches s:
  - Given an object x that hashes to the bucket $h(x)$
  - we scan buckets to the right of $h(x)$ until we find a bucket $h(s)$ to which the name of some cache s hashes. (We wrap around the array, if need be.)
  - We then designate s as the cache responsible for the object x.



CamScanner

Consistent Hashing

- Which objects x are assigned to which caches s:



Source: https://web.stanford.edu/class/cs168/l/l1.pdf

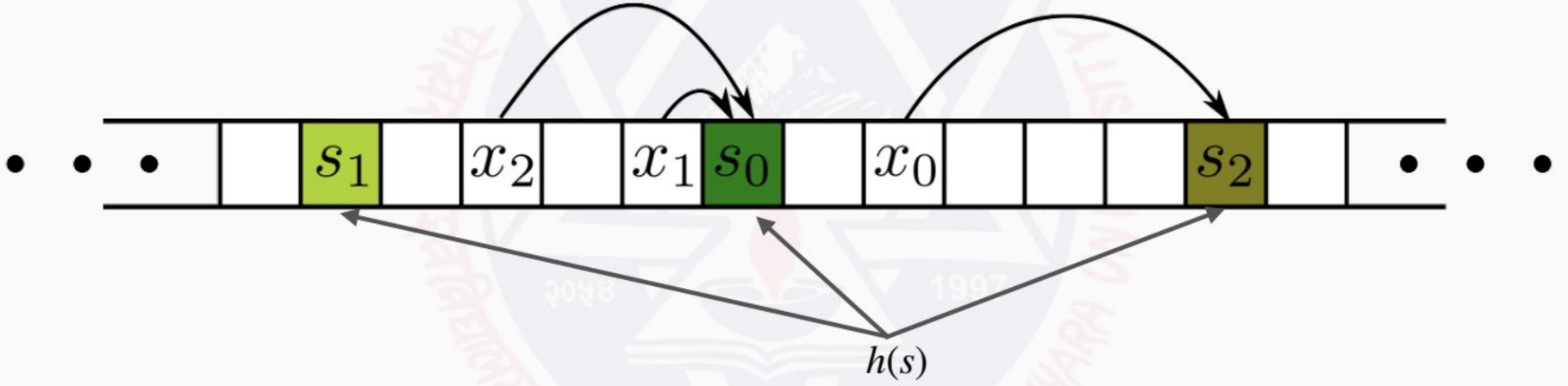
Each element of the array above is a bucket of the hash table.

Each object x is assigned to the first cache server s on its right.

CamScanner

Consistent Hashing

- Which objects x are assigned to which caches s:



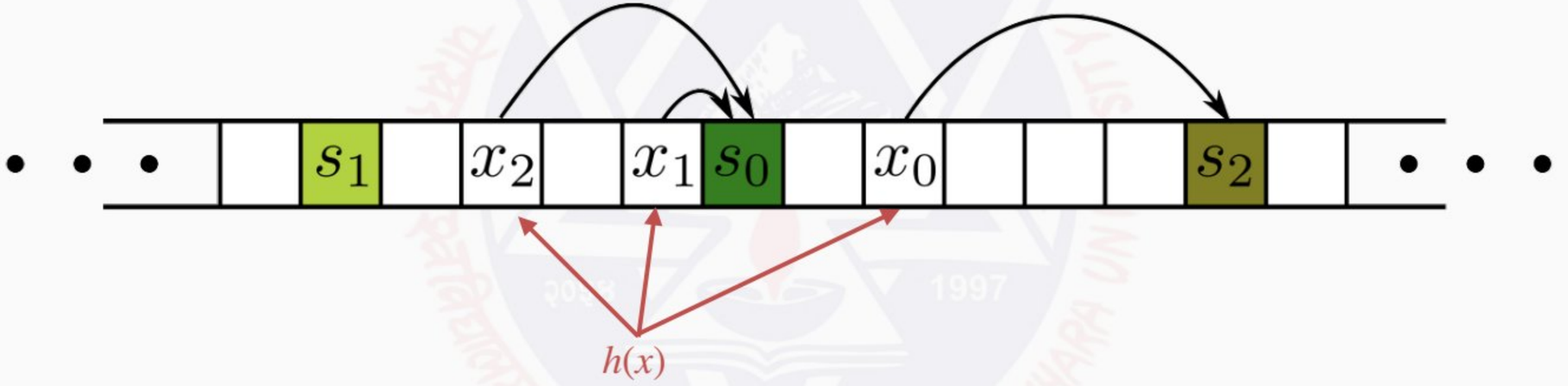
Each element of the array above is a bucket of the hash table.

Each object x is assigned to the first cache server s on its right.

CamScanner

Consistent Hashing

- Which objects x are assigned to which caches s:

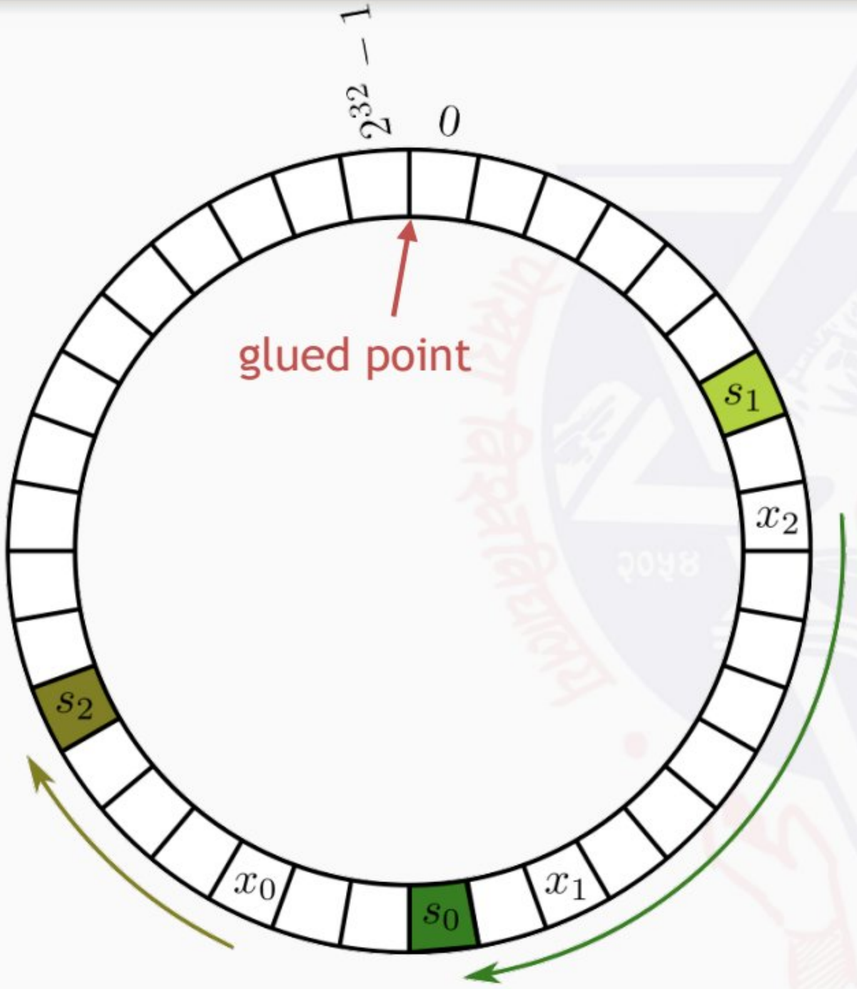


Each element of the array above is a bucket of the hash table.

Each object x is assigned to the first cache server s on its right.

CamScanner

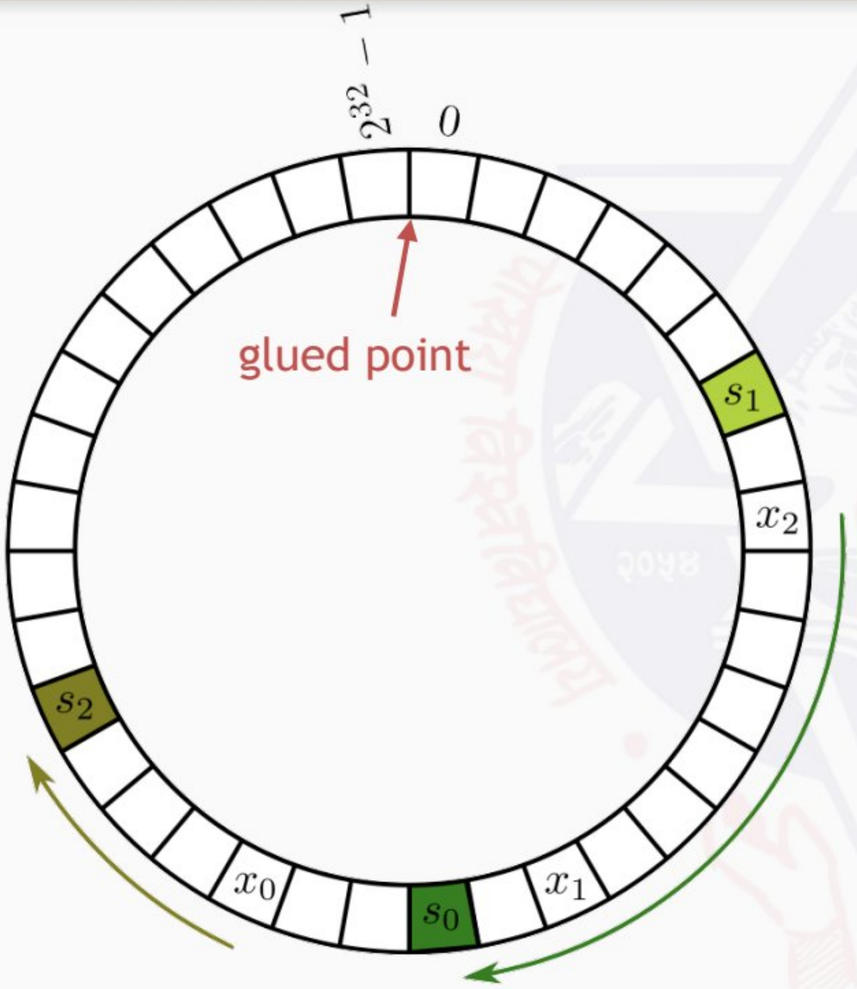
Consistent Hashing



- We glue 0 and 2^{32} – 1 together, so that objects are instead assigned to the cache server that is closest in the clockwise direction.
- This solves the problem of the last object being to the right of the last cache.

CamScanner

Consistent Hashing



- Caches and objects both hash to points on this circle
- an object is stored on the cache server that is closest in the clockwise direction.
- Thus n caches partition the circle into n segments
- each cache is responsible for all objects in one of these segments $(\frac{1}{n})$.

CamScanner

Consistent Hashing: Properties

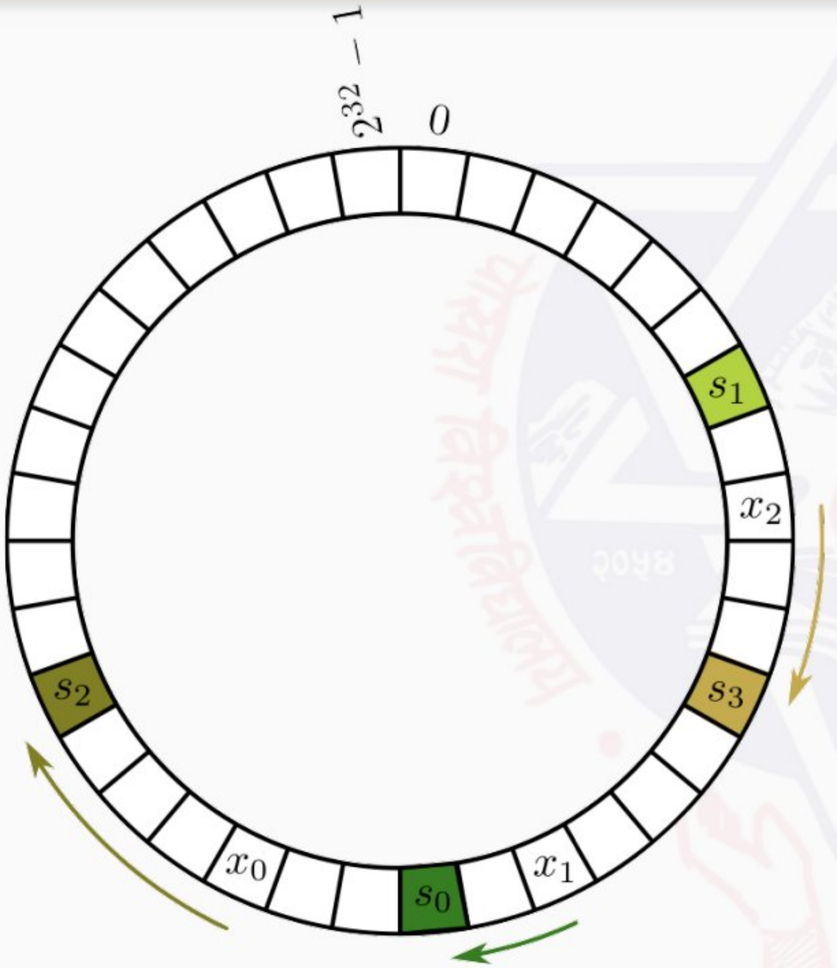
- ***Property 1:***
  - Expected load for each cache server = $\frac{1}{n}$ fraction of objects
- ***Property 2:***
  - Adding the nth cache causes only a $\frac{1}{n}$ fraction of the objects to relocate

    (in best case- it occurs if $h(s)$ evenly distribute s)
  - clearly the objects on the new cache have to move from where they were before.



CamScanner

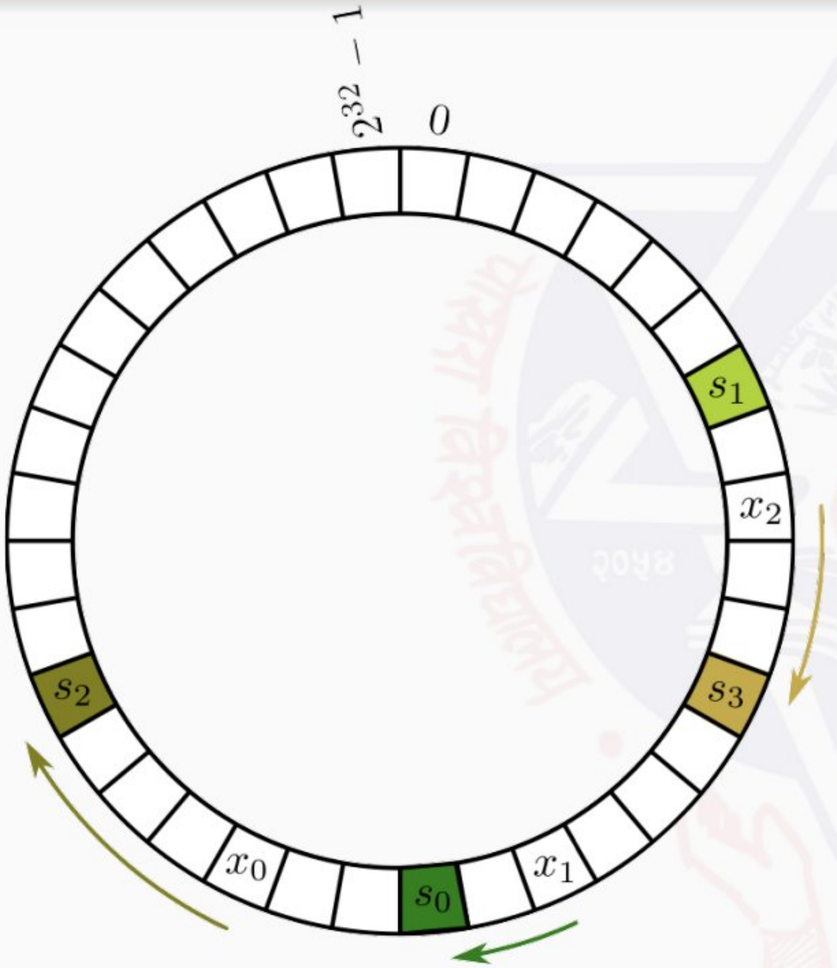
Consistent Hashing: Adding a new cache



- Adding a new cache server s_3.
  Object x_2 moves from s_0 to s_3.

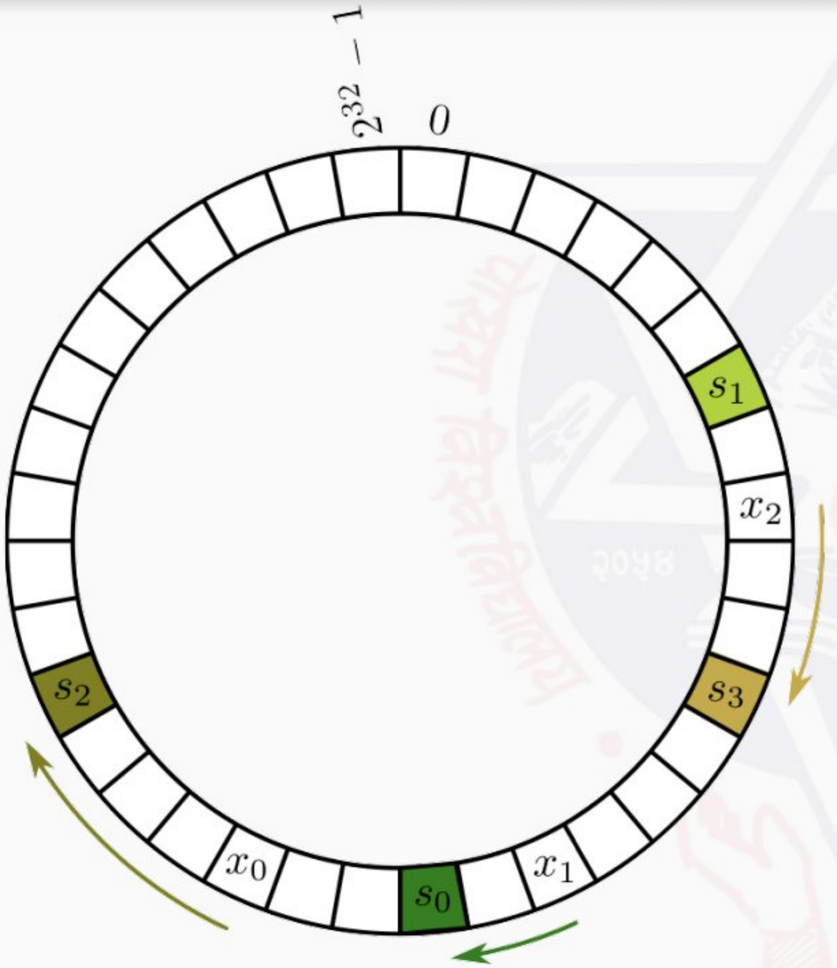
CamScanner

Consistent Hashing: Adding a new cache



- On average, adding n^{th} cache may not move whole $\frac{1}{n}$ fraction of the objects
- See only x_2 is moved to from s_0 to s_3. The x_1 is still belongs to s_0

Consistent Hashing: Removing a cache



- Opposite of adding
- To removing a cache server - say s_3, first assign all the objects (for which s_3 is responsible) to the successor cache server i.e. s_0 in our example.
- Remove s_3

Consistent Hashing: Lookup and Insert

- Given an object x,
  - we use efficient *successor* operation to lookup or insert x
- *successor* operation scans rightward/clockwise for s
  - stops at s where $h(s) \geq h(x)$
- How to make *successor* operation?
  - Use a data structure to store cache names
  - Which data structure can make *successor* operation faster



CamScanner

Consistent Hashing: Lookup and Insert

- Which data structure can make *successor* operation faster?
  - Hash table ?
  - A heap?
  - A Binary Search Tree?



CamScanner

Consistent Hashing: Lookup and Insert

- Which data structure can make *successor* operation faster?
  - ~~Hash table:~~ not good- it doesn't maintain order
  - A heap?
  - A Binary Search Tree?



CamScanner

Consistent Hashing: Lookup and Insert

- Which data structure can make *successor* operation faster?
  - ~~Hash table:~~ not good- it doesn't maintain order
  - ~~A heap:~~ not good- it maintains only for partial order (min or max at a time)
  - A Binary Search Tree?



CamScanner

Consistent Hashing: Lookup and Insert

- Which data structure can make *successor* operation faster?
  - ~~Hash table:~~ not good- it doesn't maintain order
  - ~~A heap:~~ not good- it maintains only for partial order (min or max at a time)
  - A Binary Search Tree: maintains order of elements but must be balanced
    - e.g. Red Black tree



CamScanner

Consistent Hashing: Lookup and Insert

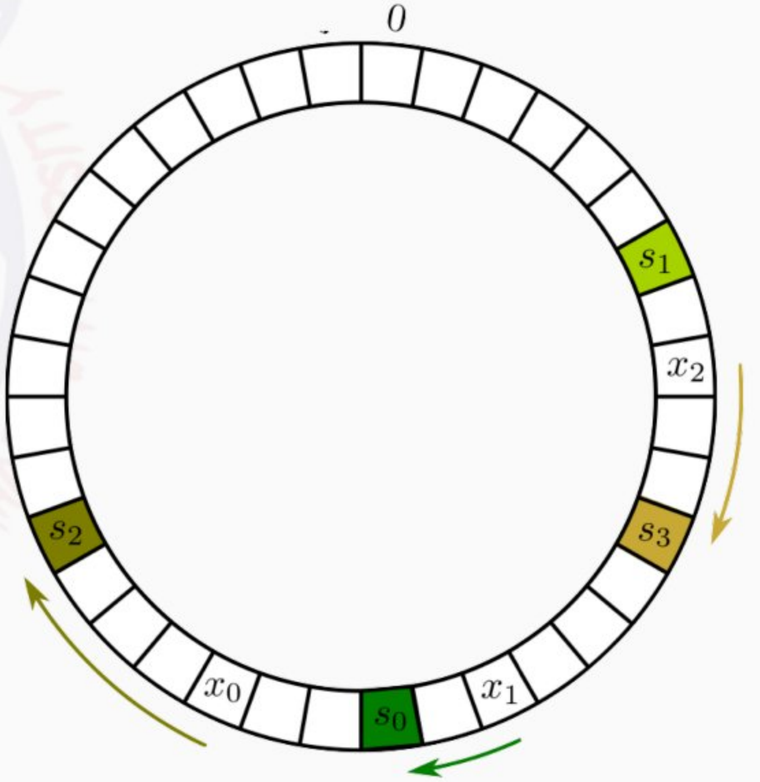
- A Binary Search Tree: maintains order of elements but must be balanced
  - e.g. Red Black tree
- Finding the cache responsible for storing a given object x
  - takes $O(logn)$ time, where n is the number of caches



CamScanner

Consistent Hashing: Reducing the variance

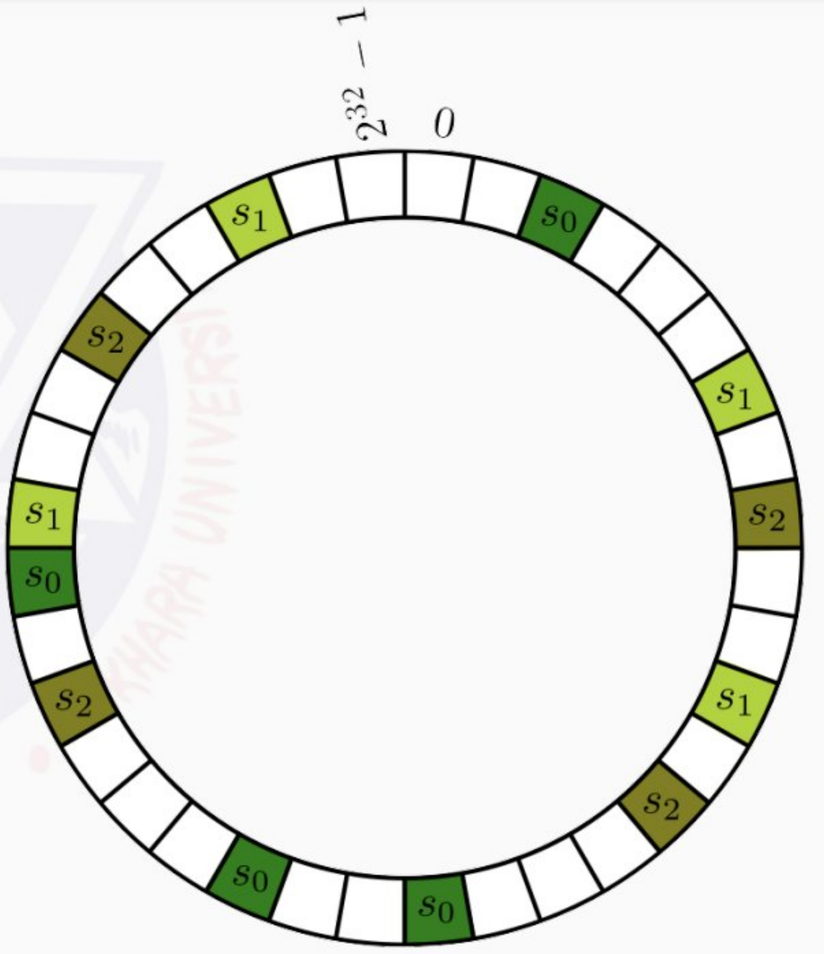
- Expected load of each cache server = $\frac{1}{n}$ fraction of objects
- But in real: load of each cache will vary
- Picking n random points on the circle, very unlikely to get a perfect partition of the circle into equal-sized segments (not uniform distribution of s)
- How to reduce this variance?



Consistent Hashing: Reducing the variance

An easy way to reduce this variance:

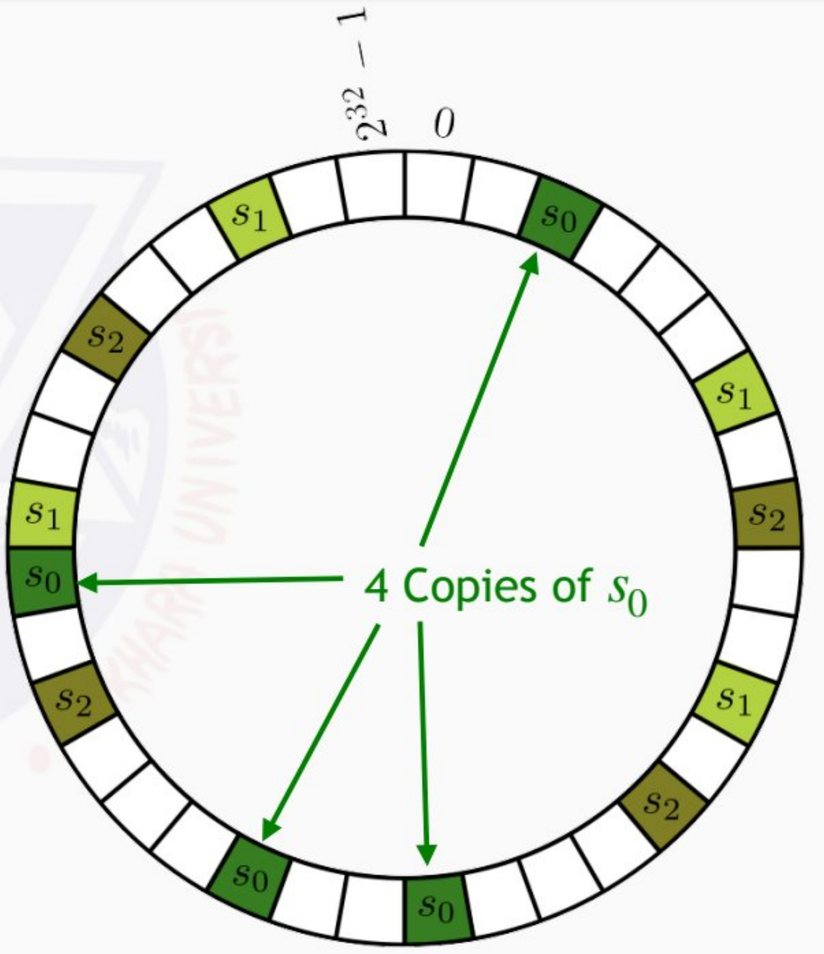
- make k "virtual copies" of each cache s,
- hash its name s with k to get $(h_1(s),\ldots, h_k(s))$
- Example: with cache servers $\{0,1,2\}$ and $k = 4$, we choose 12 points in the circle as in figure



Consistent Hashing: Reducing the variance

An easy way to reduce this variance:

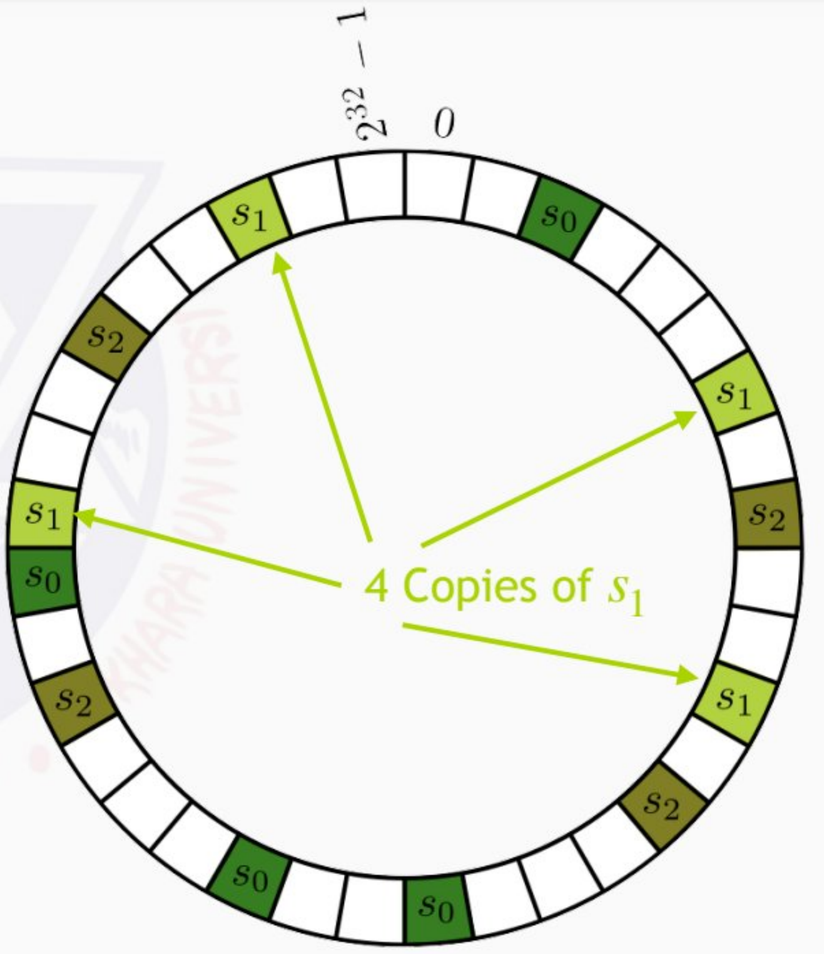
- make k "virtual copies" of each cache s,
- hash its name s with k to get $(h_1(s),\ldots, h_k(s))$
- Example: with cache servers $\{0,1,2\}$ and $k = 4$, we choose 12 points in the circle as in figure



Consistent Hashing: Reducing the variance

An easy way to reduce this variance:

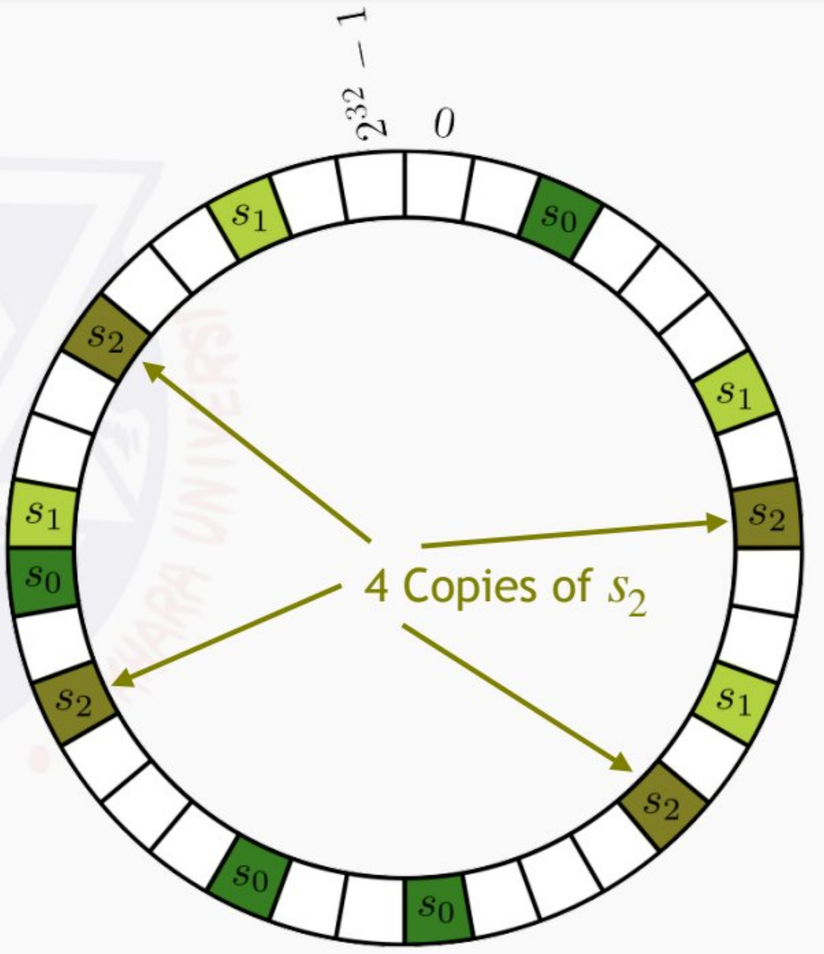
- make k "virtual copies" of each cache s,
- hash its name s with k to get $(h_1(s),\ldots, h_k(s))$
- Example: with cache servers $\{0,1,2\}$ and $k = 4$, we choose 12 points in the circle as in figure



Consistent Hashing: Reducing the variance

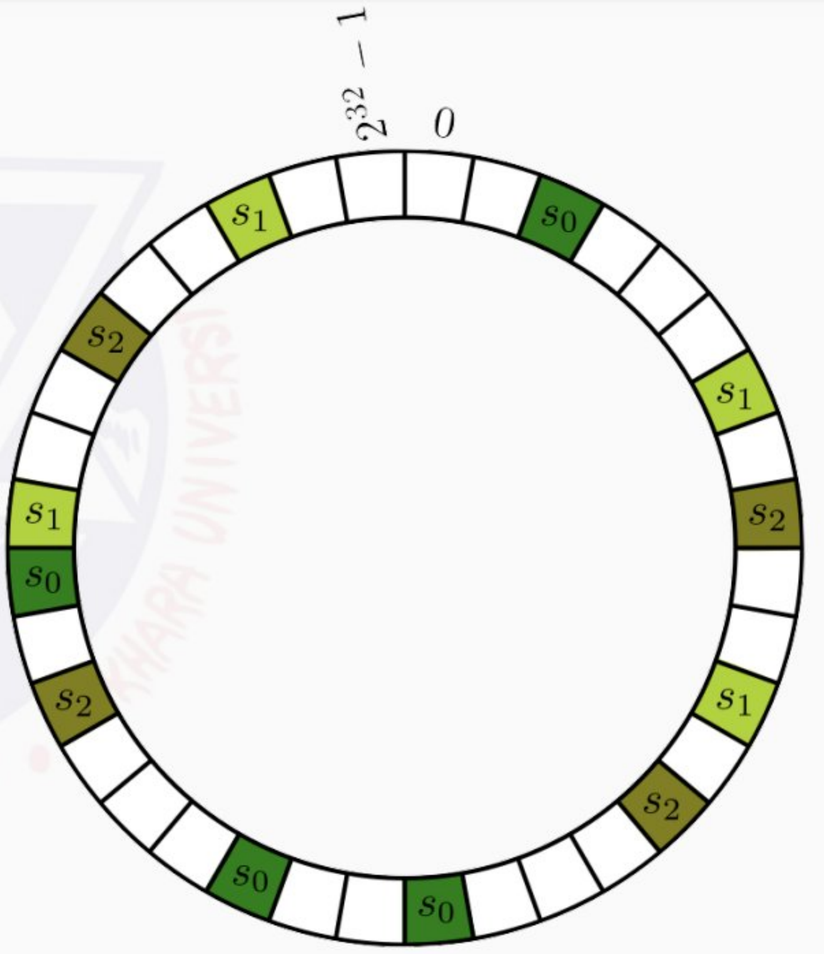
An easy way to reduce this variance:

- make k "virtual copies" of each cache s,
- hash its name s with k to get $(h_1(s),\ldots, h_k(s))$
- Example: with cache servers $\{0,1,2\}$ and $k = 4$, we choose 12 points in the circle as in figure



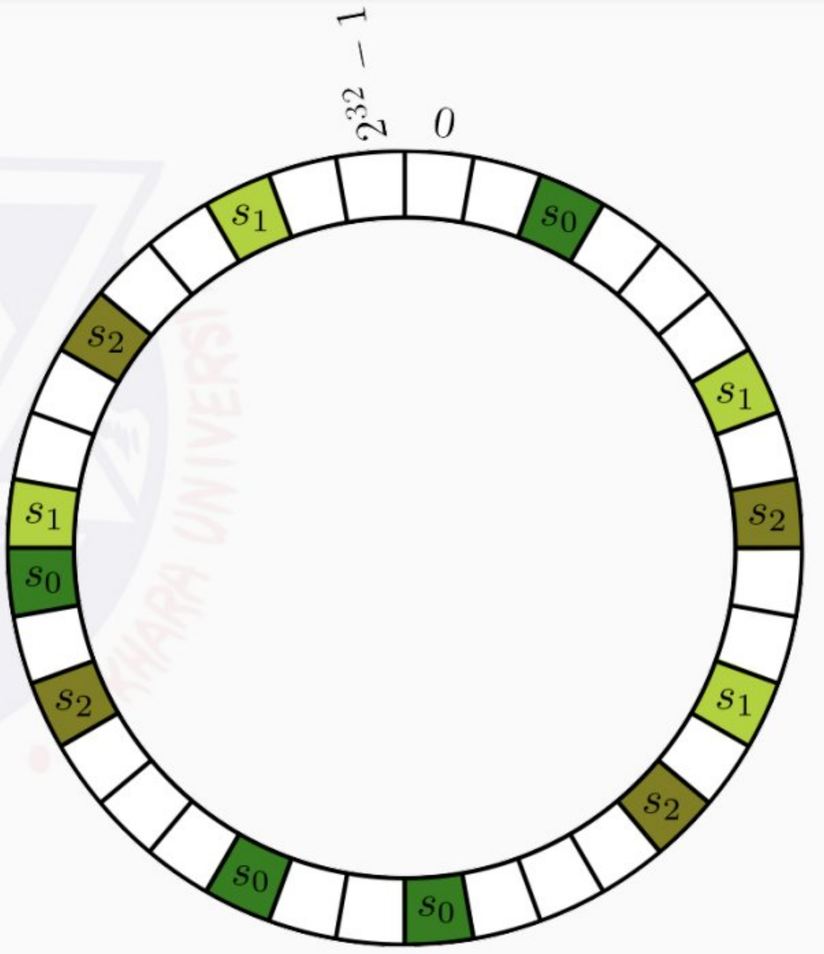
Consistent Hashing: Reducing the variance

- Objects x are assigned as before using from $h(x)$
- We scan rightward/clockwise until we encounter one of the hash values of some cache s, and
- s is responsible for storing x



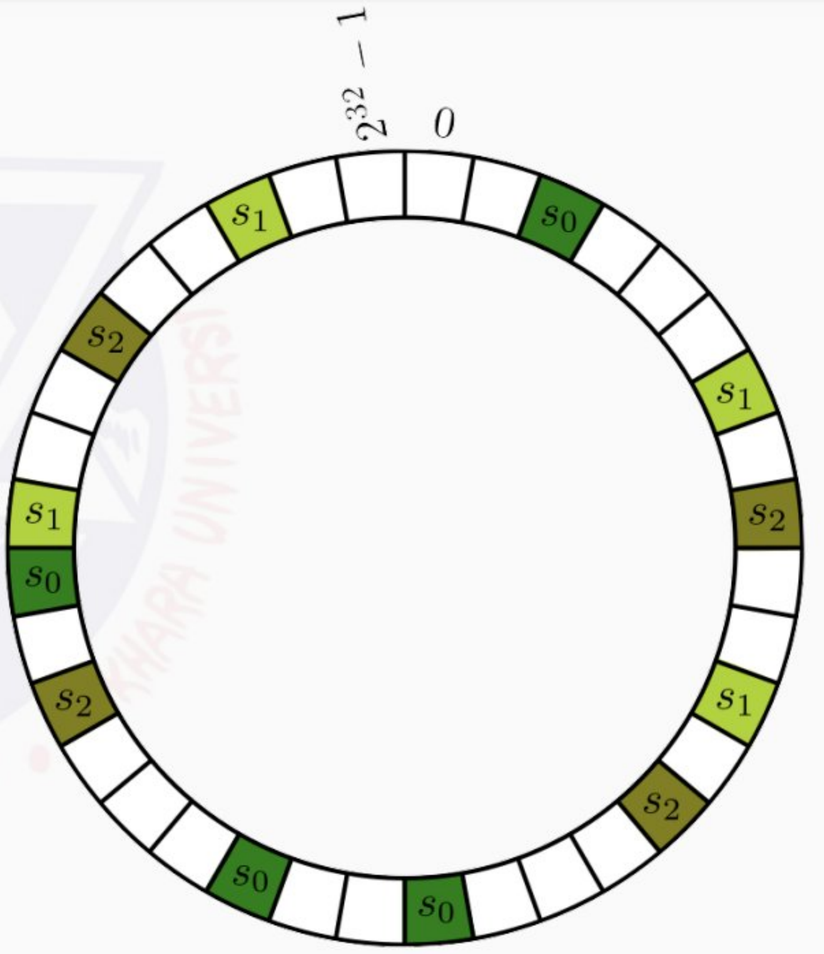
Consistent Hashing: Reducing the variance

- Still expected load = $\frac{1}{kn}$ for each s_k
- Still one cache may get more objects than expected
- Choosing $k \approx log_2 n$ is large enough to obtain reasonably balanced loads.



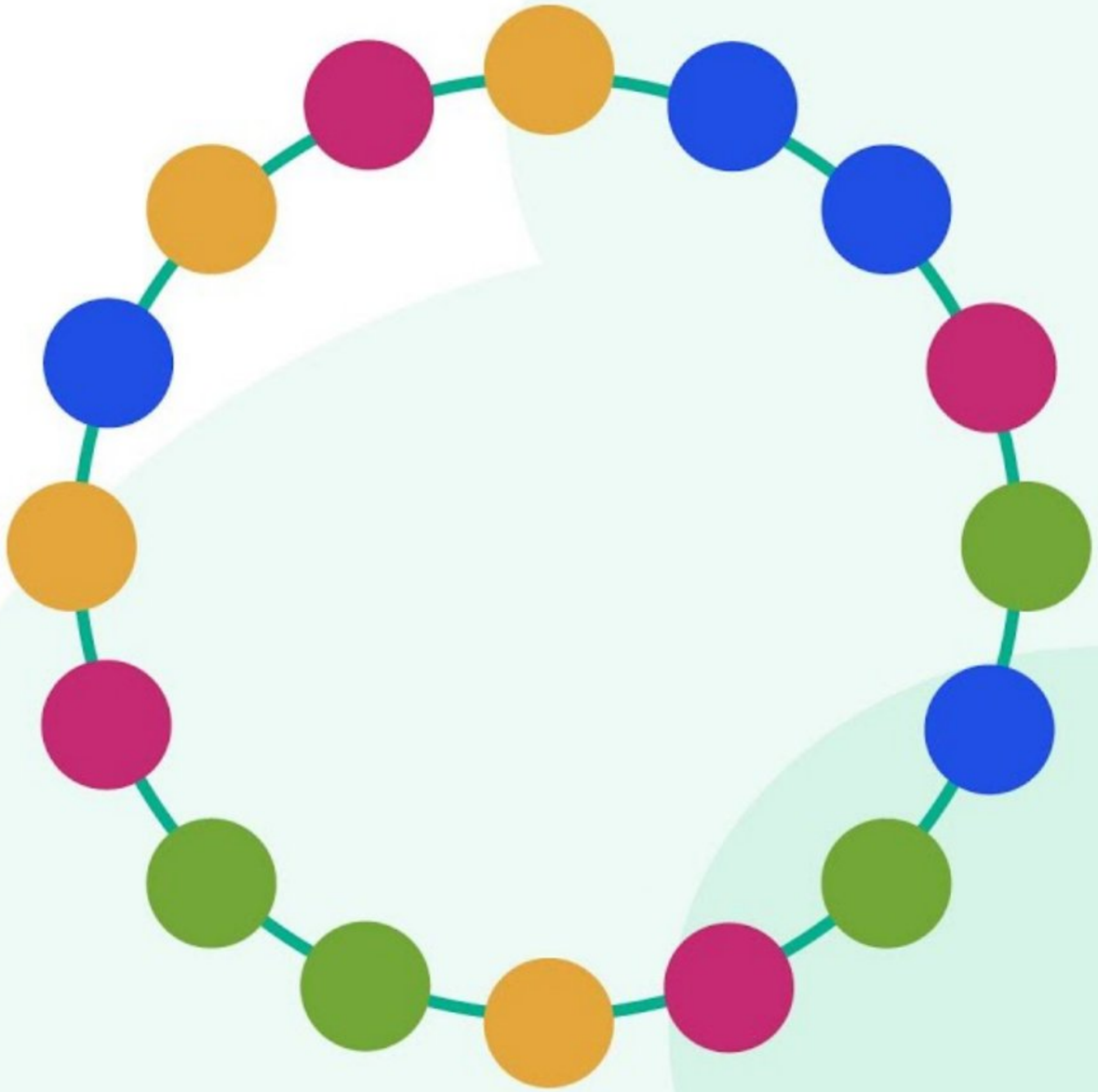
Consistent Hashing: Reducing the variance

- Virtual copies are also useful for dealing with heterogeneous caches that have different capacities.
- The sensible approach is to make the number of virtual copies of a cache server proportional to the server's capacity
- for example, if one cache is twice as big as another, it should have twice as many virtual copies



Consistent Hashing

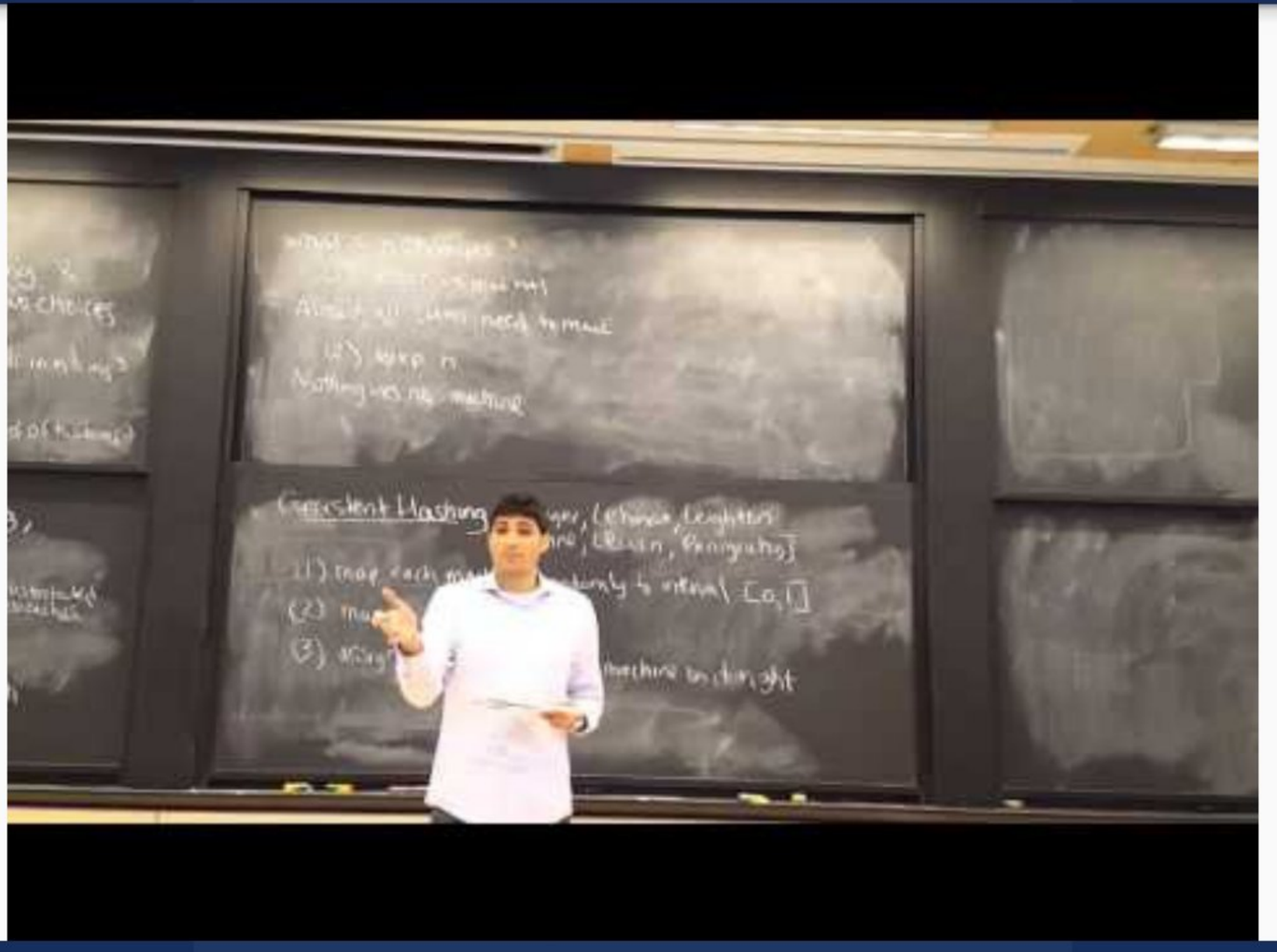
https://www.youtube.com/watch?v=UF9Iqmg94tk&t=15s



CamScanner

Consistent Hashing

https://www.youtube.com/watch?v=hM547xRIdzc



CamScanner

Hashing in Distributed System

Data distribution method of consistent hashing



Routing method of a peer- to-peer system



Distributed Hash Table

(Highly efficient method for storing data in a distributed system)

CamScanner

Distributed Hash System

- It is a decentralized system
- It uses a **distributed hash table** (DHT)
  - to store/retrieve data efficiently across multiple nodes in a network.
- It is commonly used in
  - peer-to-peer (P2P) networks
  - distributed storage systems
  - blockchain technologies
- Data is stored as $(key, value)$ pairs
  - The key is hashed to determine the node responsible for storing the $value$



CamScanner

Distributed Hash System

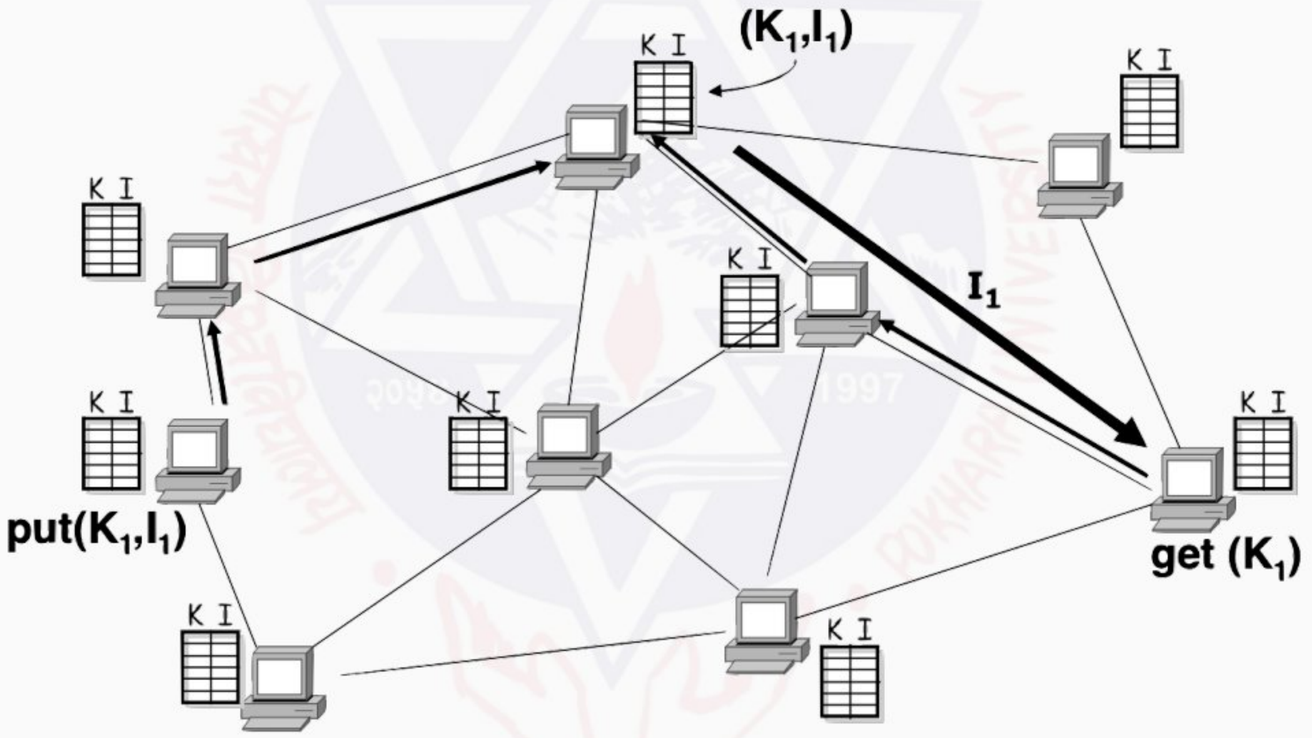
- Popular Distributed Hash Systems
  - Chord
  - Kademlia
  - Pastry
  - Tapestry



CamScanner

Distributed Hash Table (DHT)

- Hash Table Interface: $put(key, item)$ and $get(key)$



Case Studies

- ***BitTorrent and distributed hash table***
  - Hints: BitTorrent is a peer-to-peer (P2P) file-sharing protocol that allows users to download and upload files without relying on a central server. One of the key technologies that make BitTorrent decentralized is the Distributed Hash Table (DHT), which enables trackerless torrents.
- ***Amazon DynamoDB and Cord/Kademila***
  - Hints: DynamoDB is inspired by Amazon's original Dynamo system, which is a DHT-like key-value store. DynamoDB is not a pure DHT, but it borrows many principles from DHTs, especially from Amazon Dynamo. DynamoDB partitions data using consistent hashing. AWS manages virtual nodes to balance load, just like DHT overlays (e.g., Chord, Kademlia).



CamScanner

Hardware Accelerated Hashing

- CPUs
- GPUs
- FPGAs
- Case Studies

CamScanner

Hardware-Accelerated Hashing

- Hashing is a fundamental cryptographic and data-processing technique used for
  - efficient data retrieval
  - cryptographic security etc…
- Problem:
  - hashing large volumes of data using software alone = computationally expensive.
- Solution:
  - Hardware-accelerated hashing
  - How: This ***offloads*** these computations to specialized hardware,
  - Result: significantly improved performance, efficiency, and security.

Cryptographic meaning w.r.t. **Hashing**– In cryptographic hashing, the output (hash) has no direct meaning but serves as a unique identifier for input data, ensuring integrity and verification.

CamScanner

Hardware-Accelerated Hashing

- How to achieve HIGH PERFORMANCE
  - Uses dedicated hardware components
    - CPUs with SIMD instructions
    - Graphics Processing Units (GPUs)
    - FPGAs
- Specialized hardware can compute hashes significantly faster than general-purpose CPUs



CamScanner

Hardware-Accelerated Hashing

- **CPUs with SIMD (Single Instruction, Multiple Data) instructions**
  - Enable ***parallel processing*** of multiple hash computations,
    - Improve throughput
  - Contains ***hardware instructions***
    - for cryptographic acceleration
    - Example: Intel's SHA Extensions and ARM's Cryptographic Extensions for SHA-256 and AES
  - Examples:
    - Intel AVX
    - AMD SSE
    - ARM Neon etc.



CamScanner

Hardware-Accelerated Hashing

- **Graphics Processing Units (GPUs)**
  - designed for massively parallel execution
  - contain thousands of cores
  - SIMD : The same operation is applied to multiple data points simultaneously
  - **High Memory Bandwidth**: Faster memory access compared to CPUs.
  - **Specialized Compute Units (CUs)**: Designed for efficient floating-point and integer operations.
  - highly efficient for **parallel hashing tasks**
    - (***Data retrieval***, password cracking, cryptographic mining, and blockchain verification)
  - GPU Programming
    - allow developers to optimize hashing algorithms for GPUs.
    - **OpenGL** (Open Computing Language)- open source
      - (we use it to accelerate ***perfect hashing*** using GPUs)
    - **NVIDIA CUDA** (Compute Unified Device Architecture) etc…

CamScanner

Hardware-Accelerated Hashing

- **Field-Programmable Gate Array (FPGA)**
  - Semiconductor device
    - allows users to customize hardware functionality after manufacturing (Reconfigurable).
    - Direct control over hardware logic (Low Latency)
  - **hardware-level parallelism** (Unlike CPUs and GPUs)
    - massively parallel
    - can be reprogrammed multiple times.
  - It consists **programmable logic blocks** and **reconfigurable interconnects**
    - For instance- they can be **reconfigured** for specific hashing algorithms
  - Energy Efficient
    - Custom hardware consumes **less power** than CPUs/GPUs.
  - FPGA Programming
    - Hardware Description Languages (HDLs)- Verilog, VHDL, SystemVerilog
    - High-Level Synthesis (HLS) tools like Xilinx Vivado HLS, Intel OpenCL, HDL Coder allow FPGA development using C, C++, and OpenCL.

CamScanner

Case Studies

- Take the example of perfect hashing (we already implemented in simple CPUs machine. Now, perform a detail case study to accelerate the perfect hashing using GPUs. You can use **OpenGL** (Open Computing Language) since it is an open source framework for GPU programming. (Covers Hardware-Accelerated Hashing, GPUs)
- Perform a detail case study in FPGA along with its one application either in
  - hashing
  - ML Acceleration
  - Cryptography & Blockchain.
- Submit these case studies as assignments.

CamScanner



THANK YOU

CamScanner